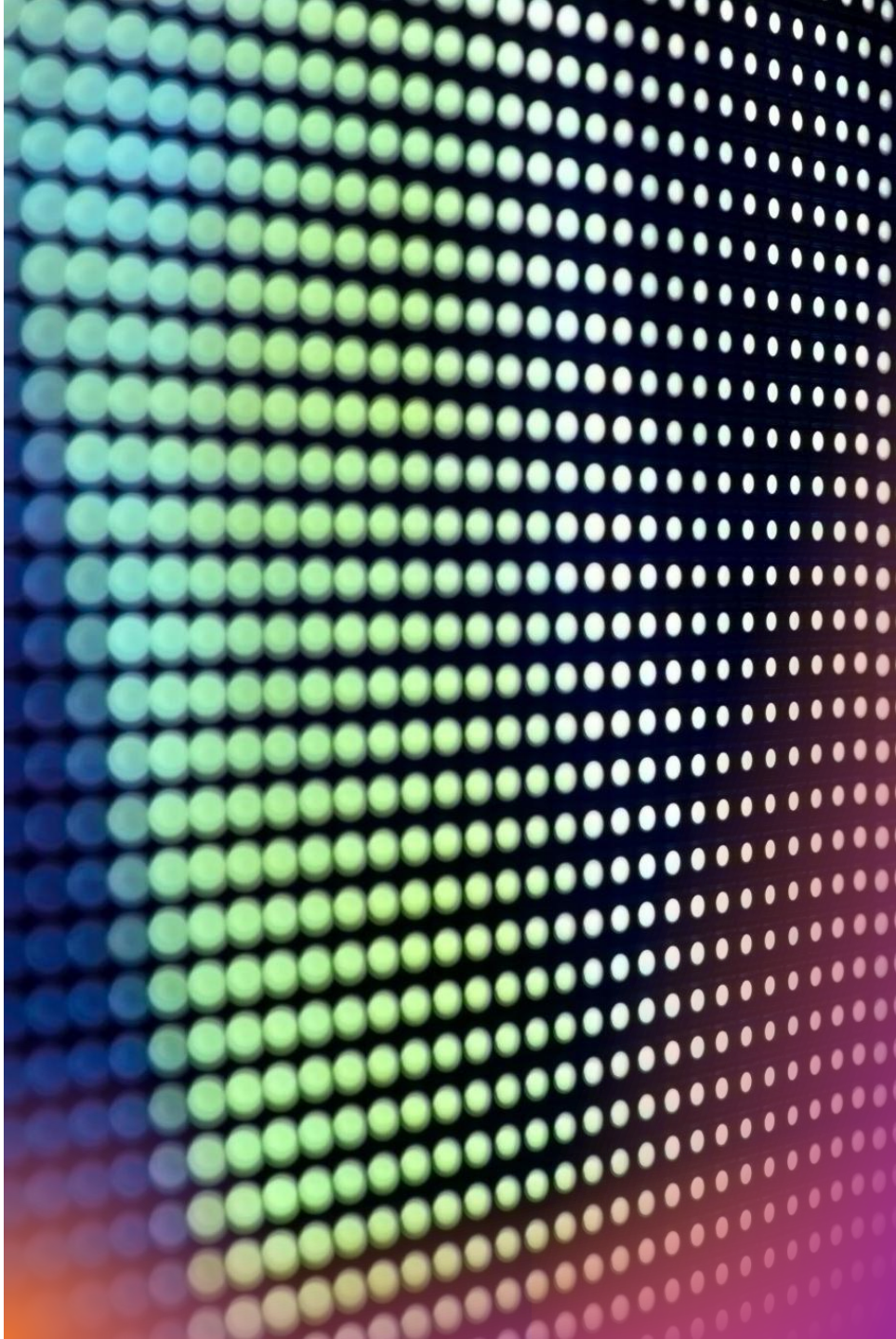
Digital Image Processing

Image Segmentation

Prof Jamshid Dehmeshki



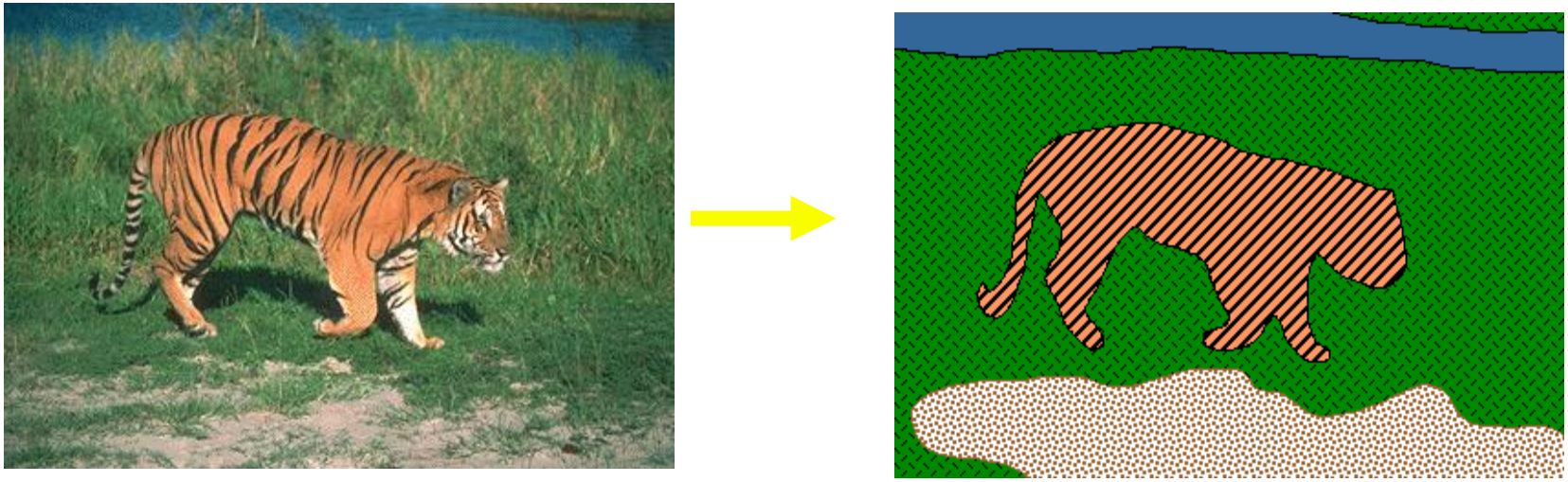
Introduction

- Image processing methods such as *histogram equalisation*, and *convolution* are methods of enhancing the **visibility** of important structure in an images for purposed of human inspection.
- **Segmentation** is the process of extracting information from an image (and often stored in a different type of data structure) e.g. locating and measuring tumours in an X-ray image or inspecting machined parts in industrial scenes.

What Is Image Segmentation?

- **Image segmentation** is the process of partitioning an image into meaningful regions (segments). The goal is to group pixels together based on similarity criteria such as intensity, color, or texture. Segmentation is often an initial step in many image processing pipelines (e.g., identifying objects in medical images, detecting shapes in computer vision, etc.).
- **Binary segmentation:** Separate the foreground (object) from the background. For instance, after thresholding an image, you get a binary image where pixels are either “1” (foreground) or “0” (background).
- **Multiple-region segmentation:** Split an image into more than two segments, e.g., labeling different tissue types in a medical scan.

Image Segmentation



Application: Motion Detection



(a) Original frame, (b) background image,(c) The thresholded difference image, (d) Detected objects (blobs).

Thresholding methods

Simple Thresholding

- A global threshold T is chosen. Each pixel is compared against this value:
  - If pixel value > T, set to max (e.g., 255)
  - Else, set to 0
- It's best used when the background and object have very different intensities.
- **Types & Behavior:**
  - THRESH_BINARY: Object = white, background = black
  - THRESH_BINARY_INV: Inverse of above

Thresolding *pseudocode*

```
% Apply the threshold
 for i=1:rows
    for j=1:cols
       if (gry(i,j)>=T)
          out(i,j) = 1;
       else
          out(i,j) = 0;
       end
    end
 end
```

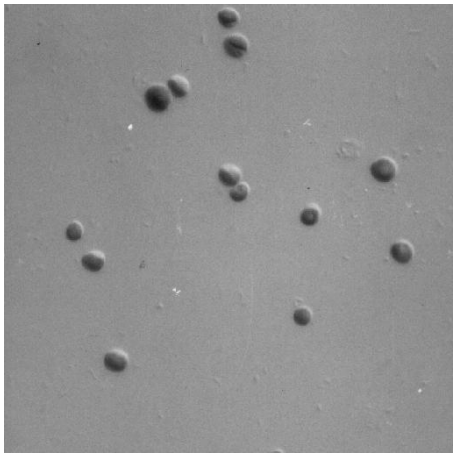
Apply threshold T to image gry to get result out.

If image pixel values are in the range 0..1, then the threshold must be also.

Make sure your images really are in this range and not 0..255.

Introduction

- Thresholding is essentially a pixel classification process in which pixels are classified on the basis of their grey-levels (perhaps including grey-levels of surrounding grey-levels) into *classes*, e.g. foreground and background in thresholding.



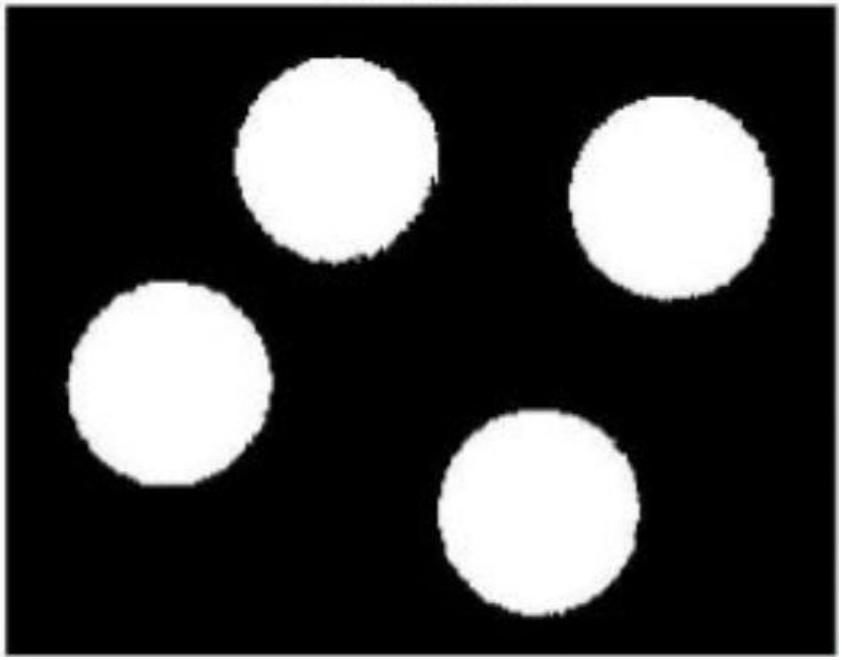
Grey-level image



Binary image

Example



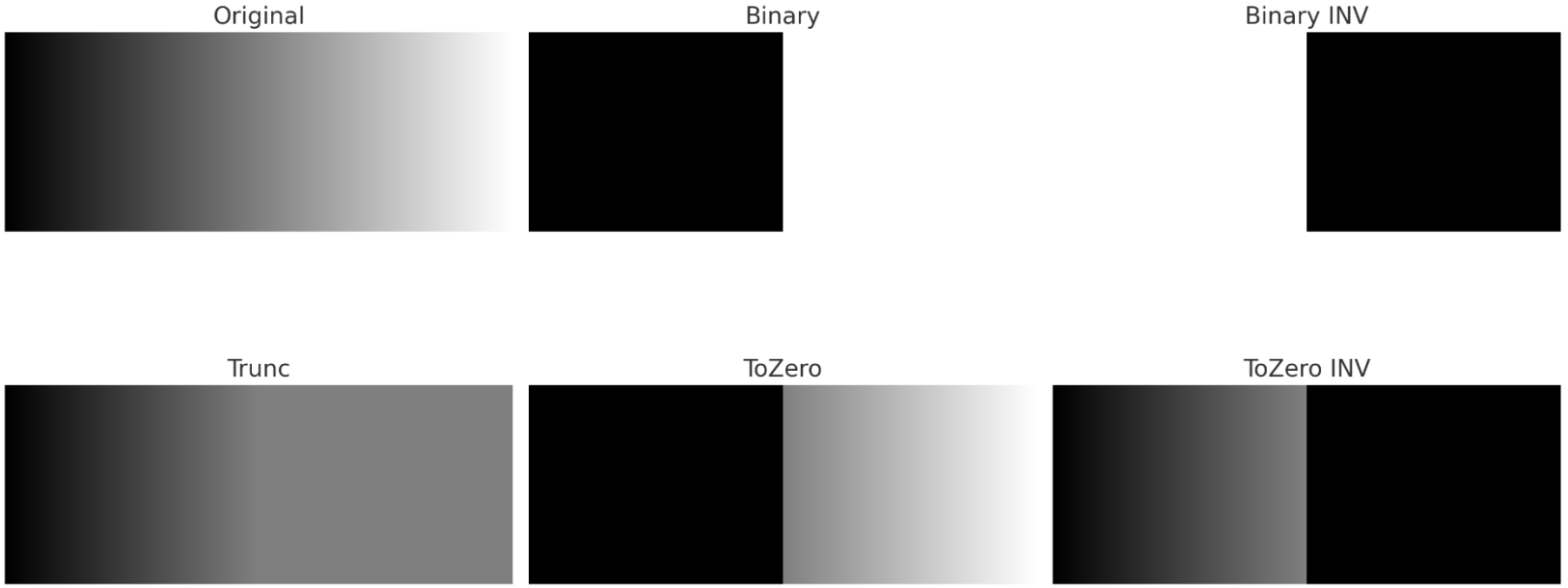




Original Image



Thresholded and segmented Image

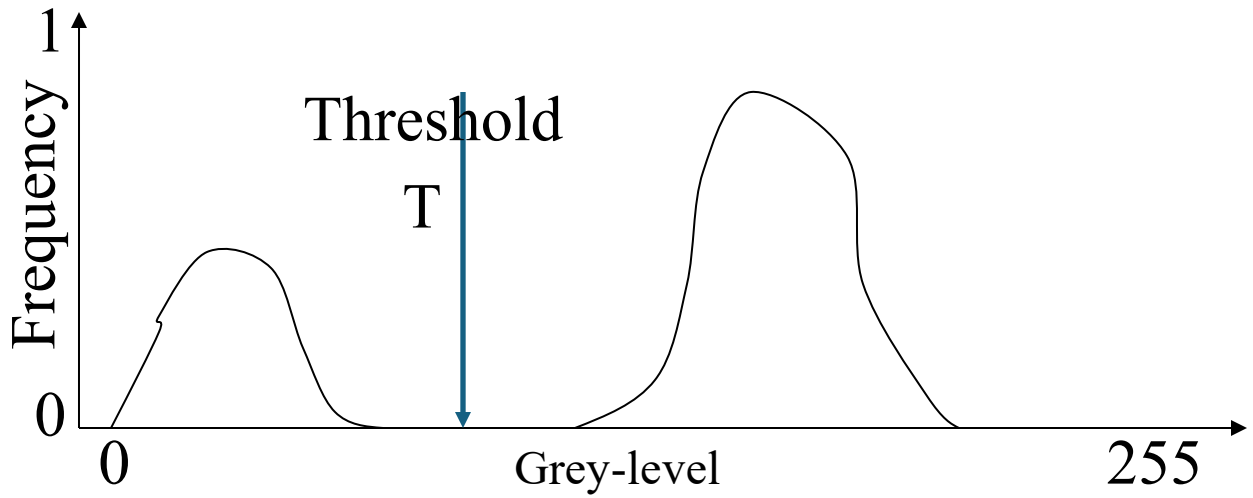


Thresholding

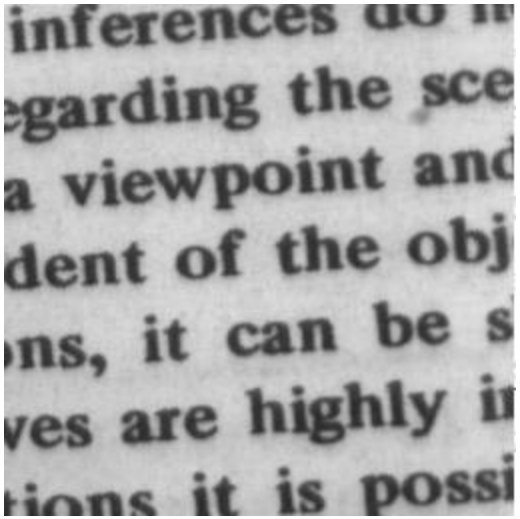
- Here’s a visualization of **Simple Thresholding** on a synthetic gradient image:
- **Original**: Left-to-right grayscale gradient.
- **Binary**: All values >127 become 255, others become 0.
- **Binary INV**: Inverse of Binary.
- **Trunc**: Values above 127 are clipped to 127.
- **ToZero**: Values ≤127 become 0; others stay.
- **ToZero INV**: Values >127 become 0; others stay.

Thresholding

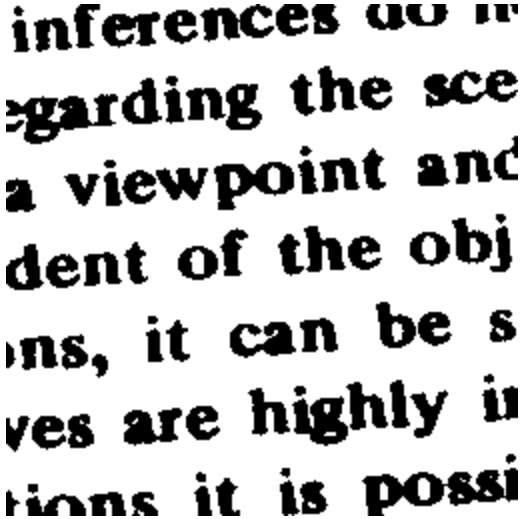
- Classification (i.e. segmentation) success depends on choosing an appropriate threshold value.



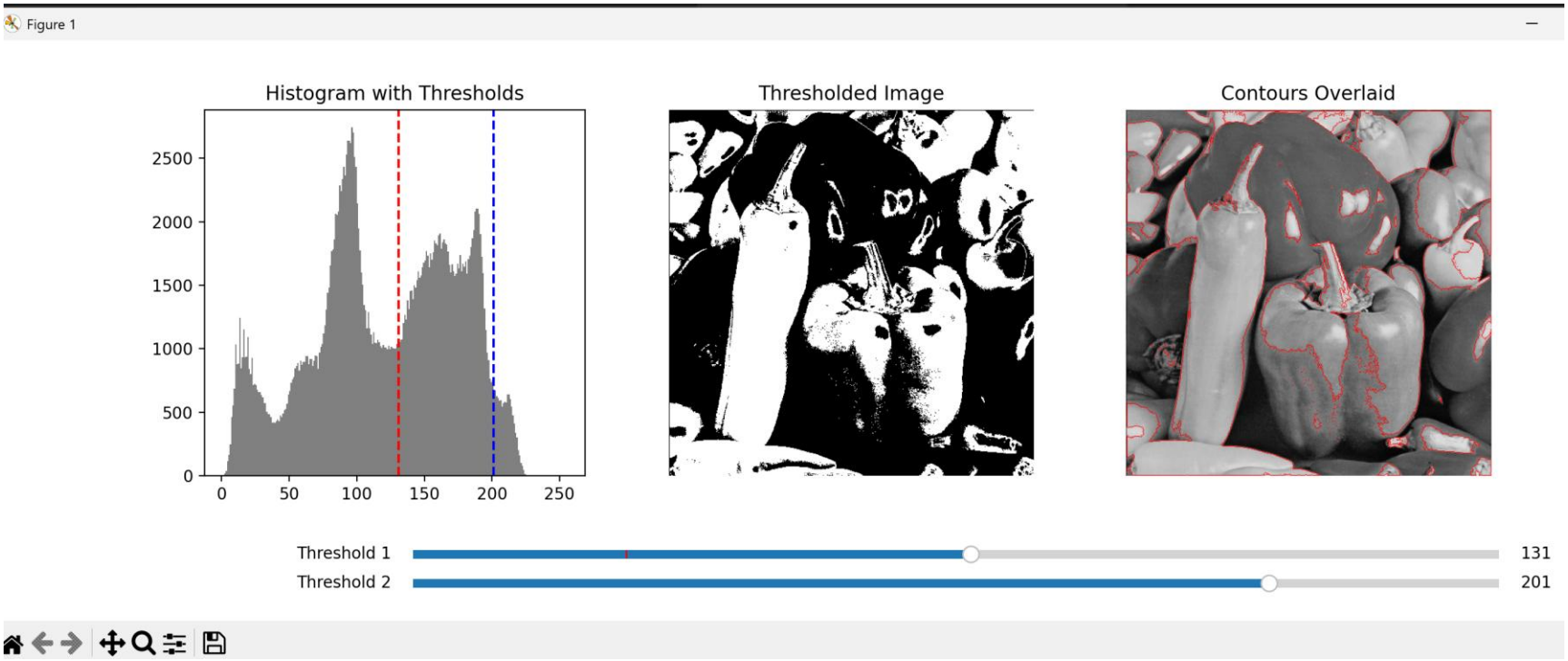
Document processing

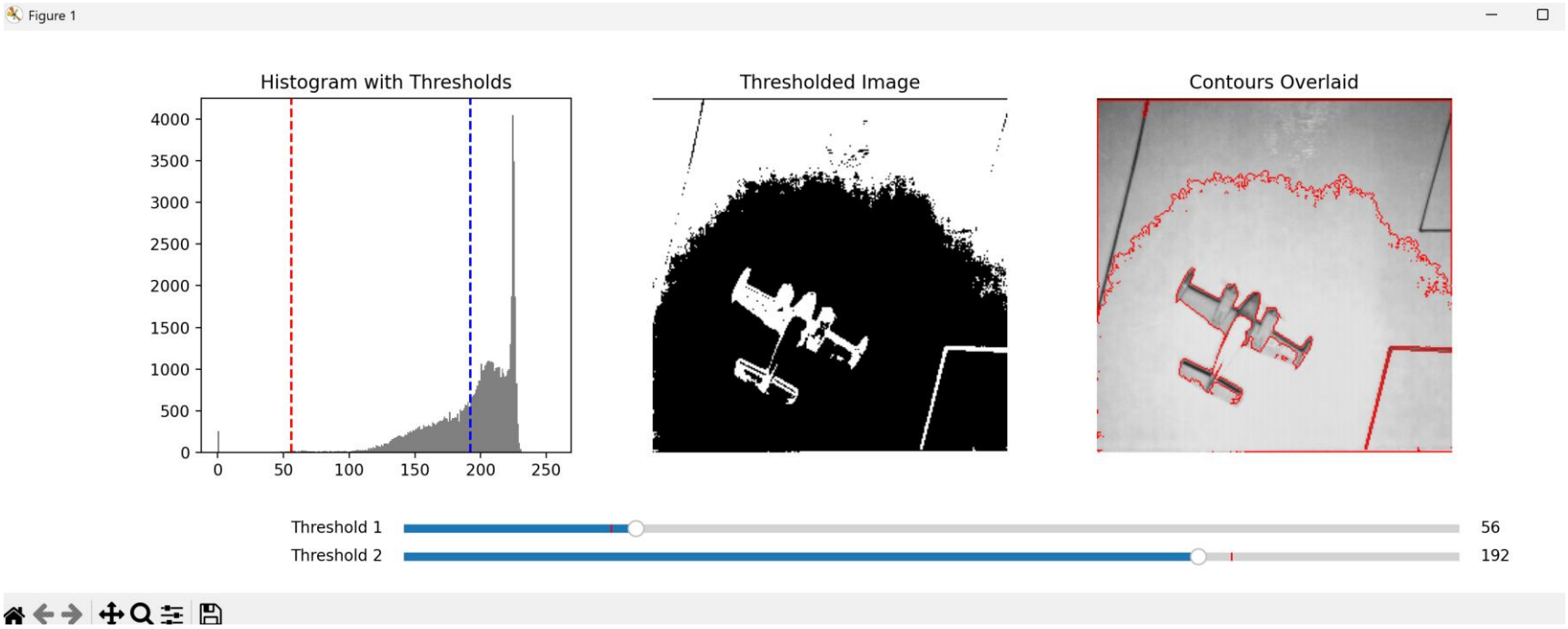


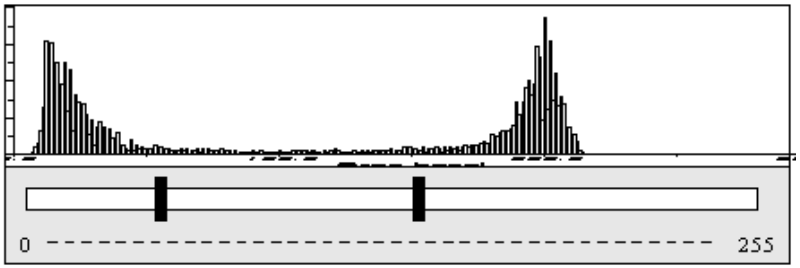
Original image

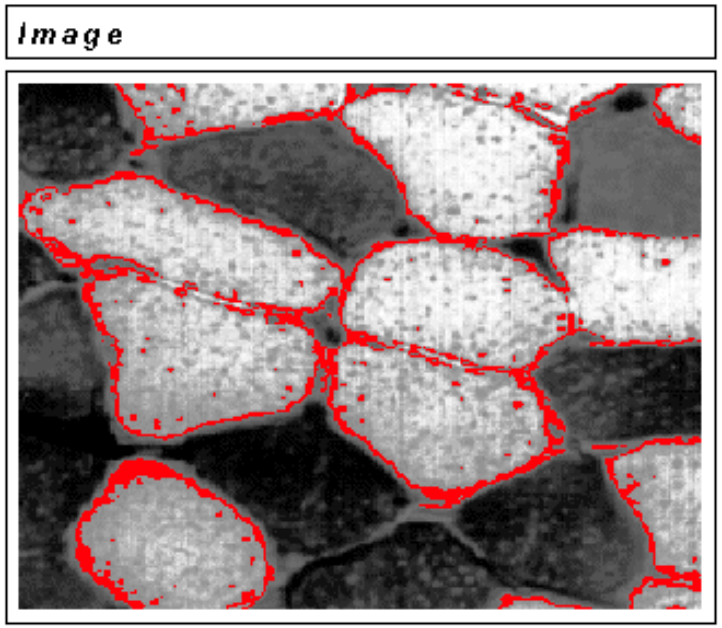


Segmented (or thresholded) image

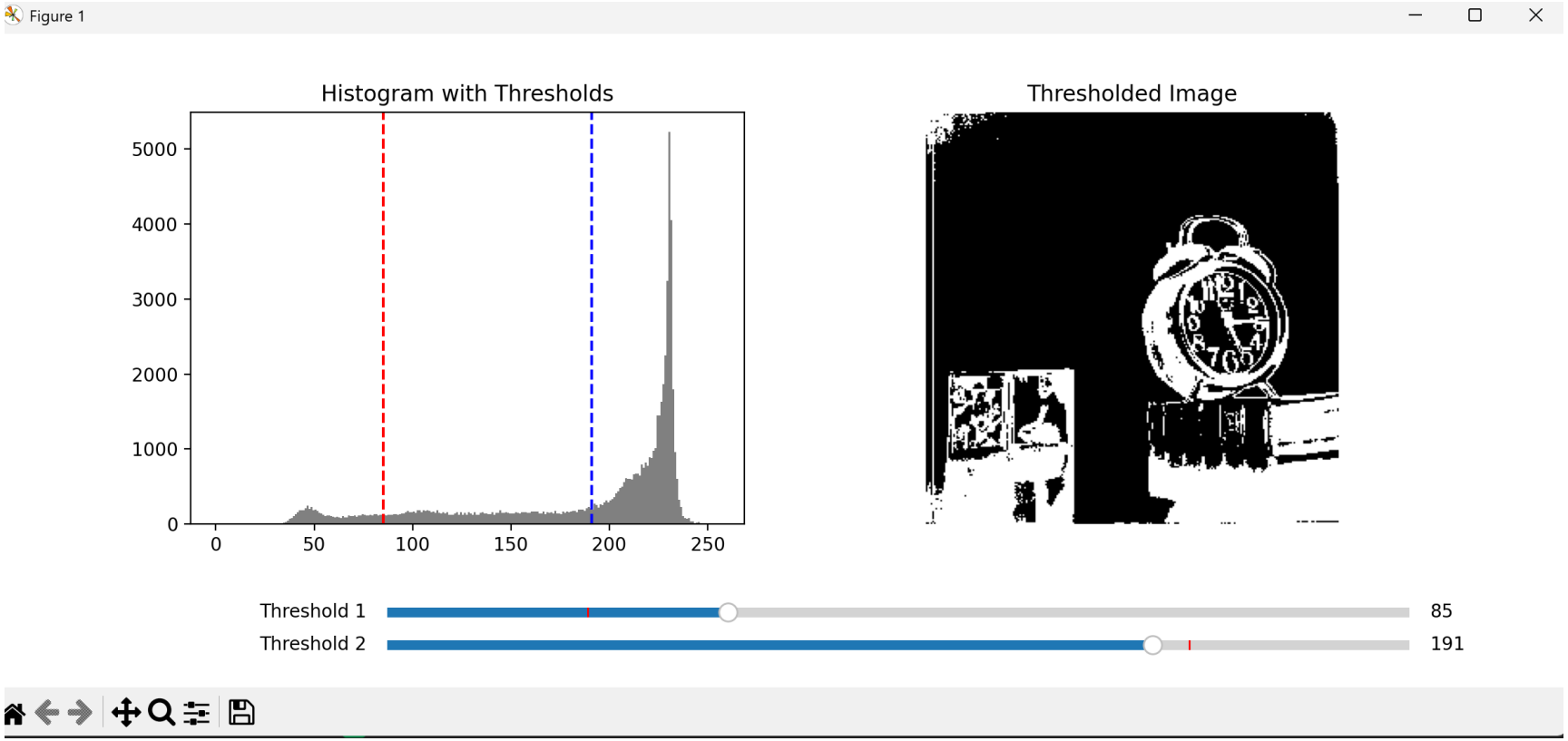

Figure 1
Histogram with Thresholds
2500
2000
1500
1000
500
0
0
50
100
150
200
250
Thresholded Image
Contours Overlaid
Threshold 1
131
Threshold 2
201

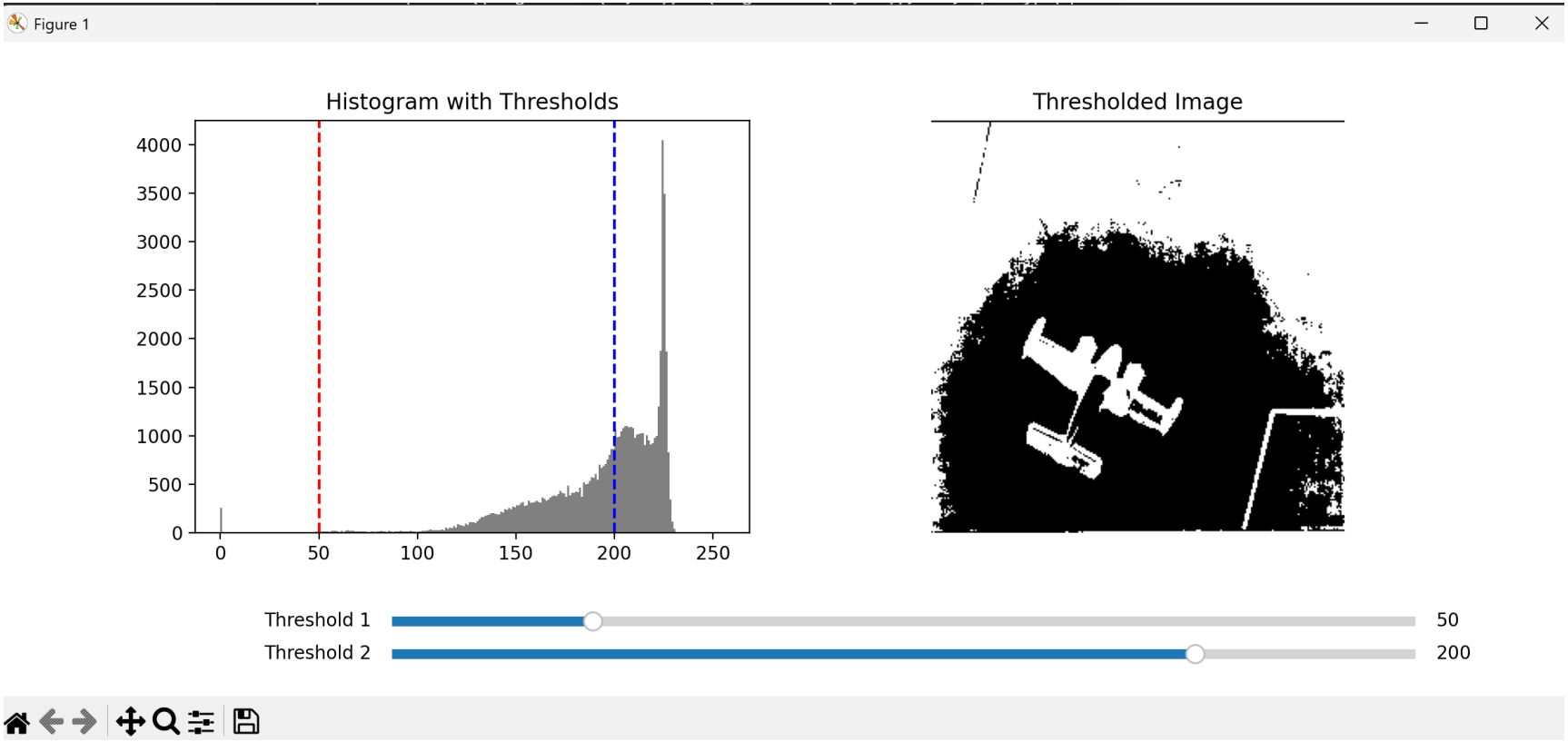

Figure 1
Histogram with Thresholds
4000
3500
3000
2500
2000
1500
1000
500
0
0
50
100
150
200
250
Thresholded Image
Contours Overlaid
Threshold 1
56
Threshold 2
192

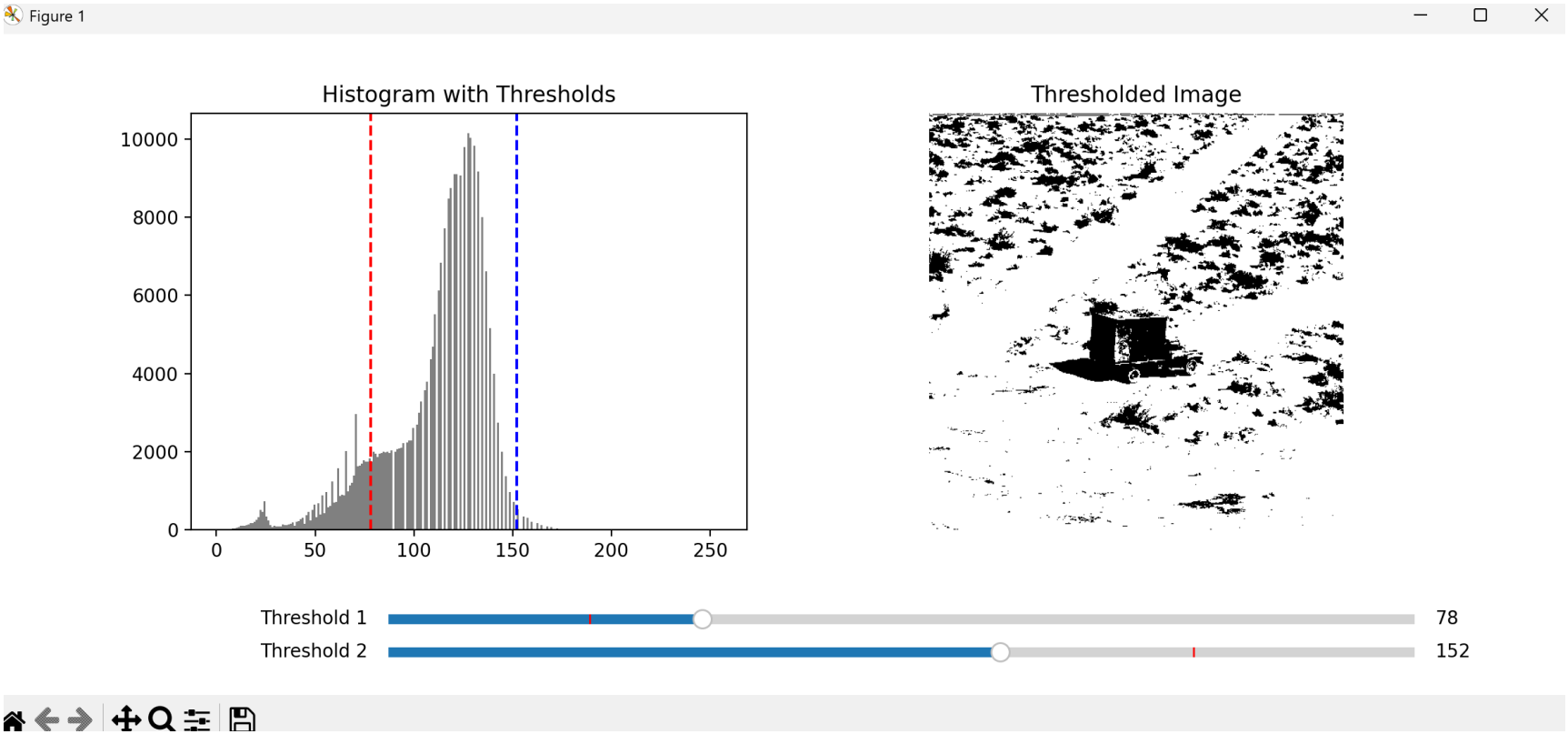


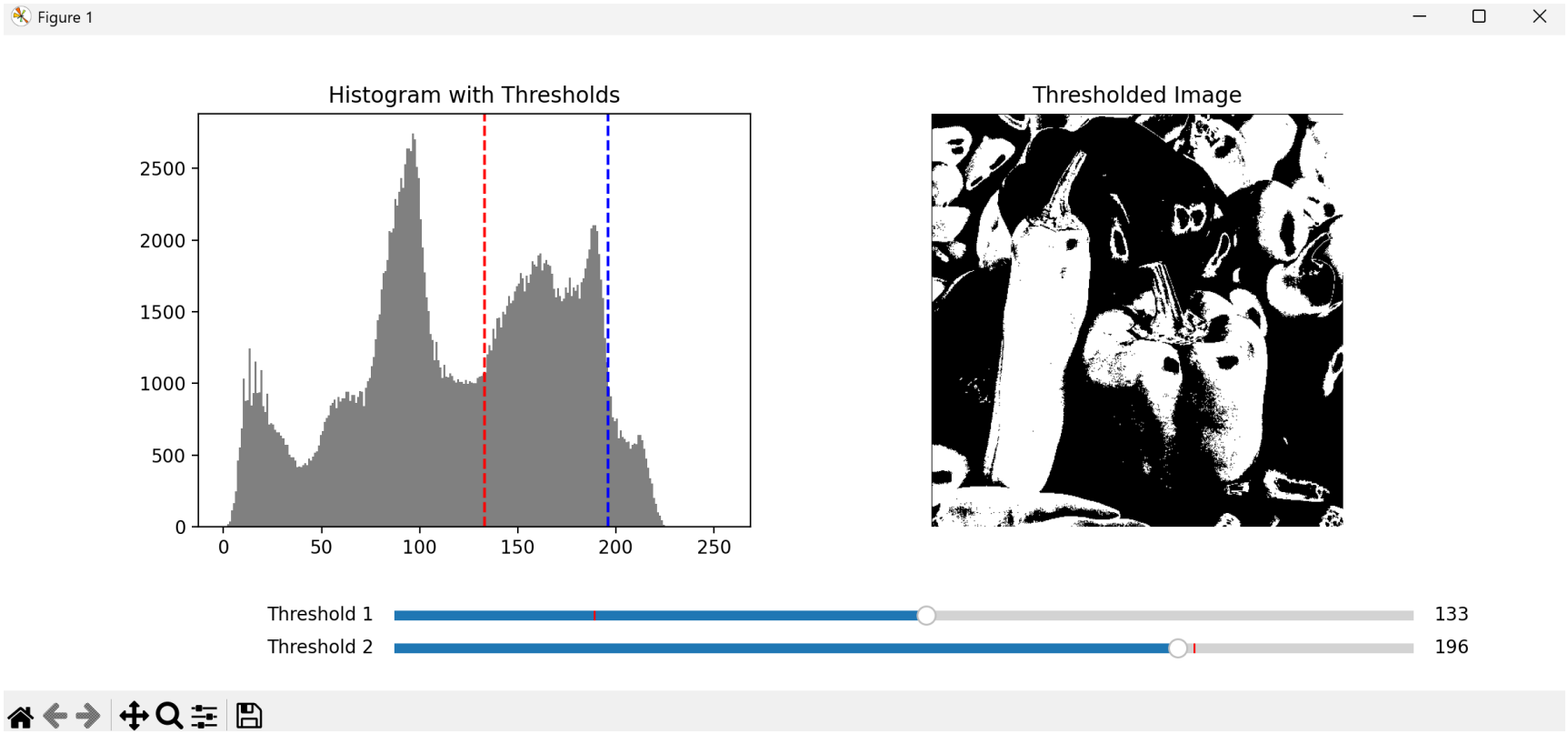


Multilevel and Interactive Thresholding


Figure 1
Histogram with Thresholds
5000
4000
3000
2000
1000
0
0
50
100
150
200
250
Thresholded Image
Threshold 1
85
Threshold 2
191


Figure 1
Histogram with Thresholds
4000
3500
3000
2500
2000
1500
1000
500
0
0
50
100
150
200
250
Thresholded Image
Threshold 1
50
Threshold 2
200

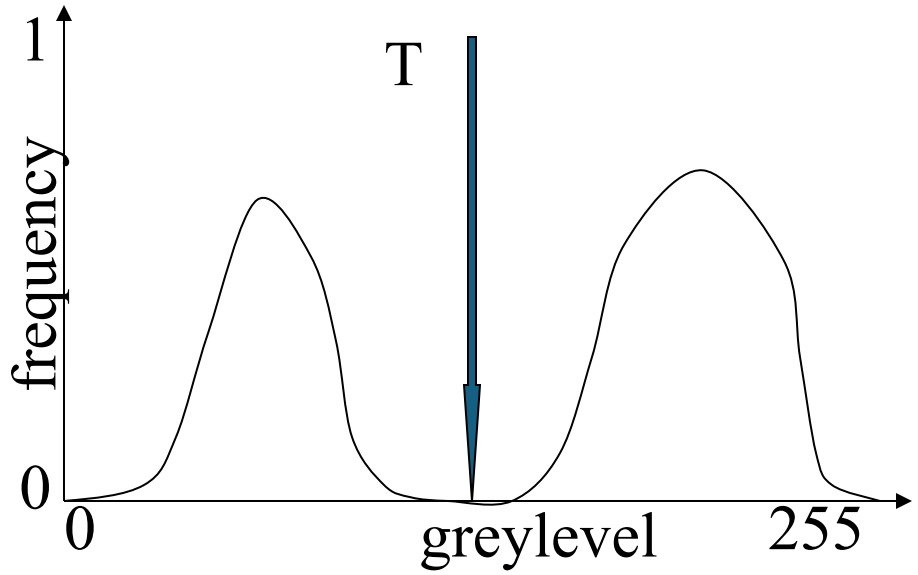

Figure 1
Histogram with Thresholds
10000
8000
6000
4000
2000
0
0
50
100
150
200
250
Thresholded Image
Threshold 1
78
Threshold 2
152


Figure 1
Histogram with Thresholds
2500
2000
1500
1000
500
0
0
50
100
150
200
250
Thresholded Image
Threshold 1
133
Threshold 2
196

Binary Segmentation

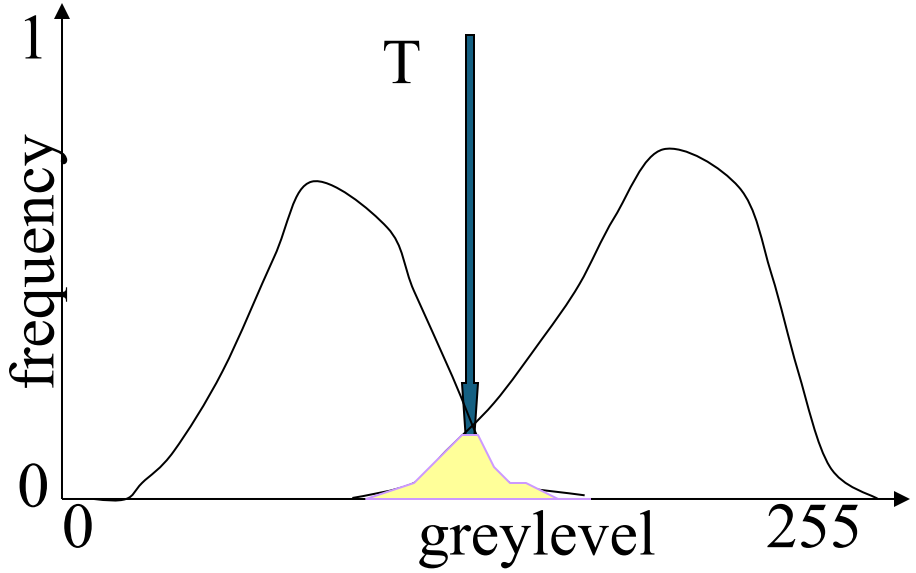
- Binary segmentation assumes:
  - that pixels associated with “features” and “background” belong to separate, non-overlapping greylevel distributions - BIMODAL distributions
  - a greylevel threshold exists between the two distributions that can partition the two sets of pixels

Histogramming problems



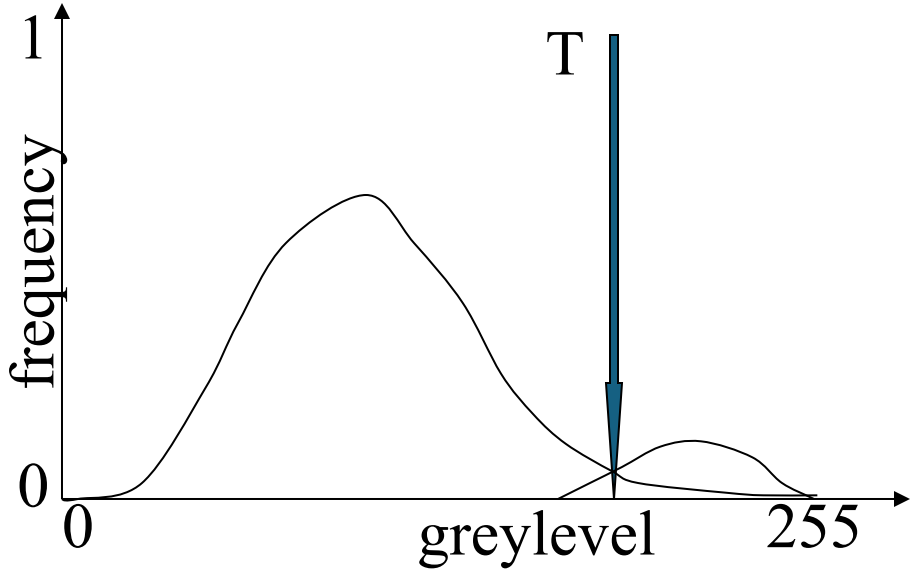
Ideal non-overlapping bimodal distribution

Histogramming problems



Overlapping distributions - pixels labelled wrongly

Histogramming problems

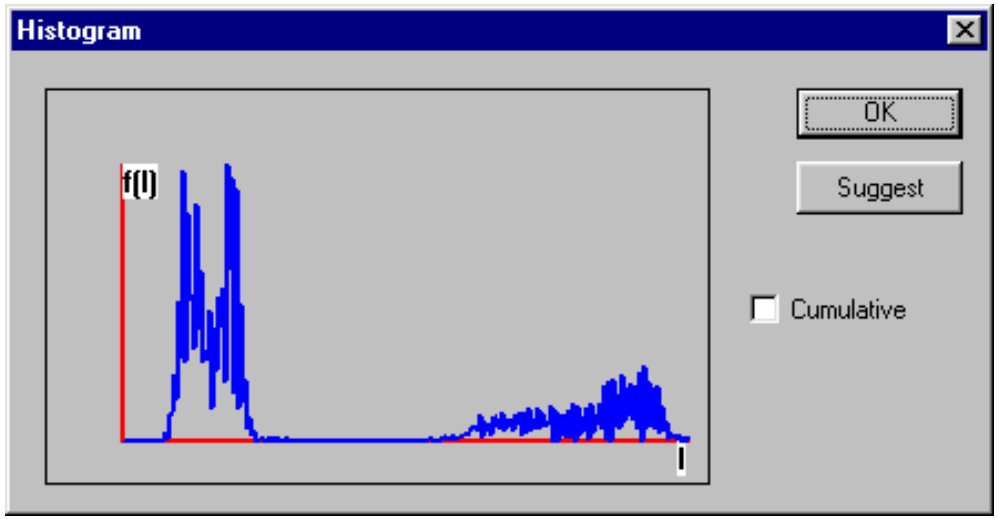


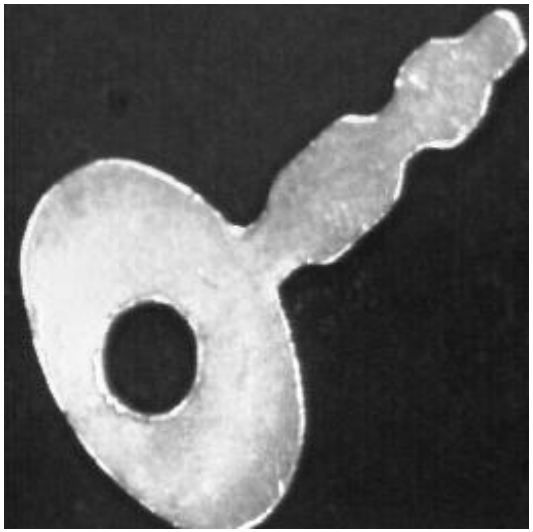
Small second distribution

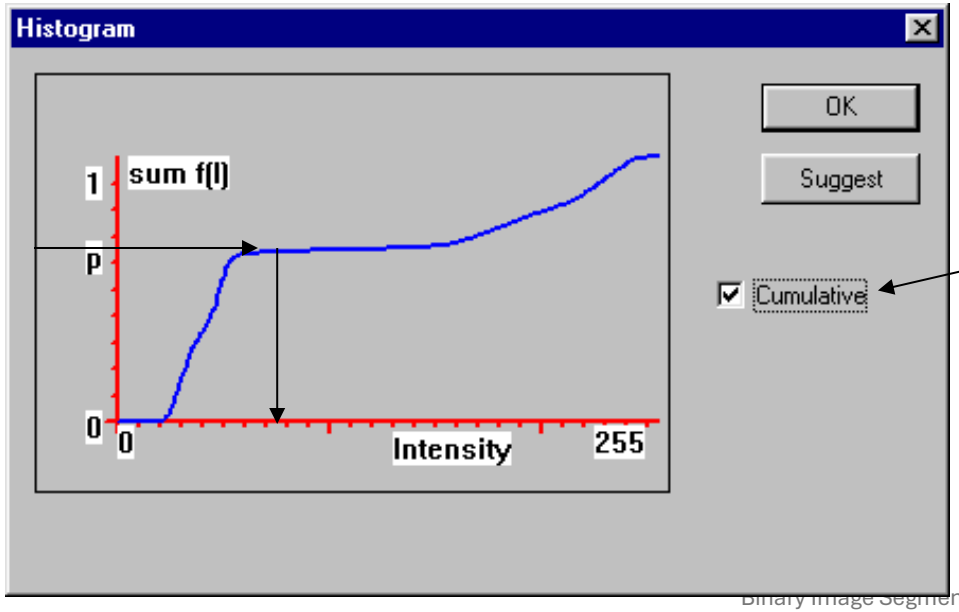
Thresholding techniques using *a priori* knowledge

- In some applications, the proportion of the image which may be labelled as feature and background may be known beforehand, e.g. text files (assuming font type, no gaps)
- The proportion of background pixels = p where $0 < p < 1$

Using *a priori* knowledge



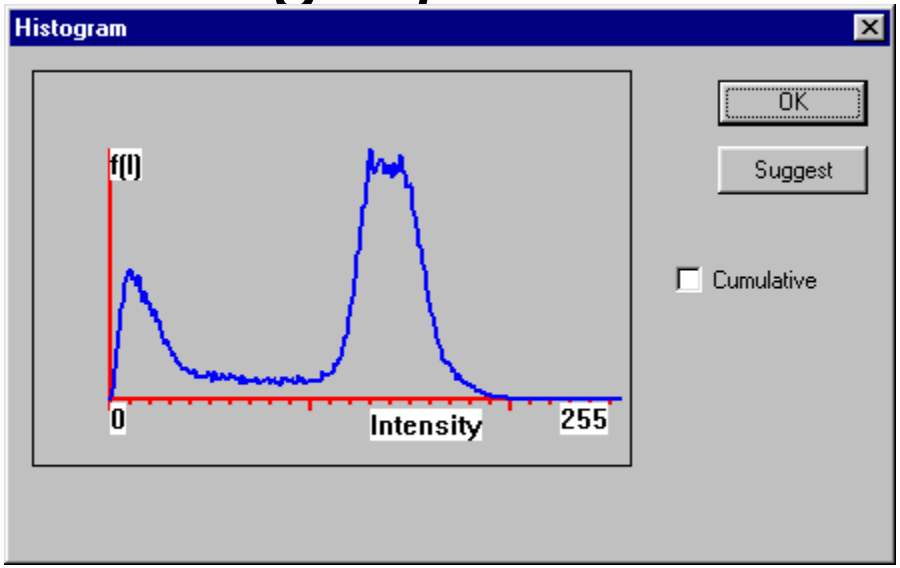


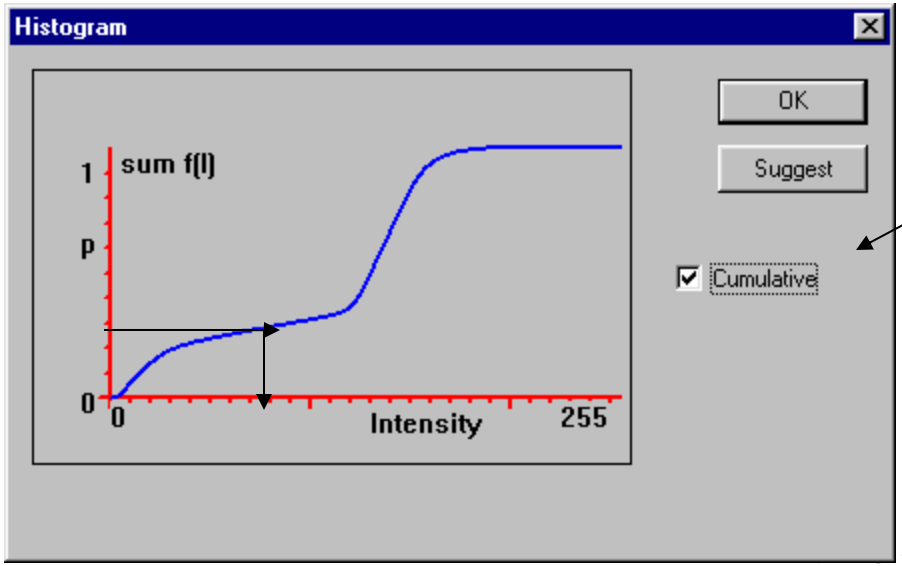


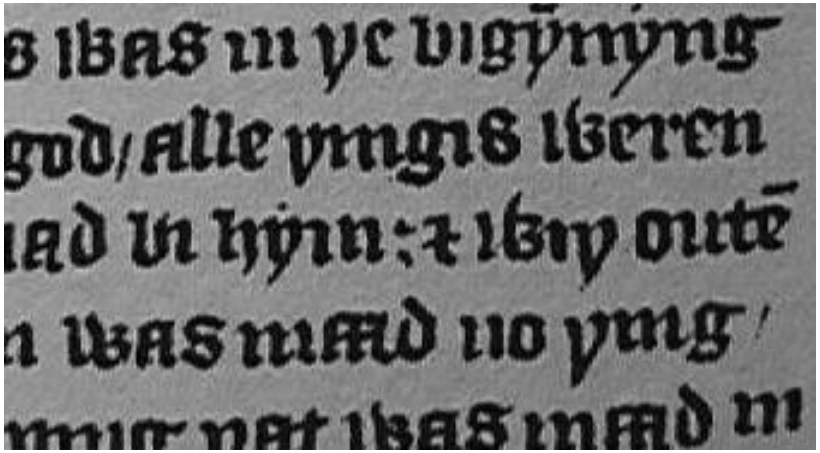
Cumulative histogram.

p = 0.7 would be used for images of this type. For this image T = 100

Using *a priori* knowledge







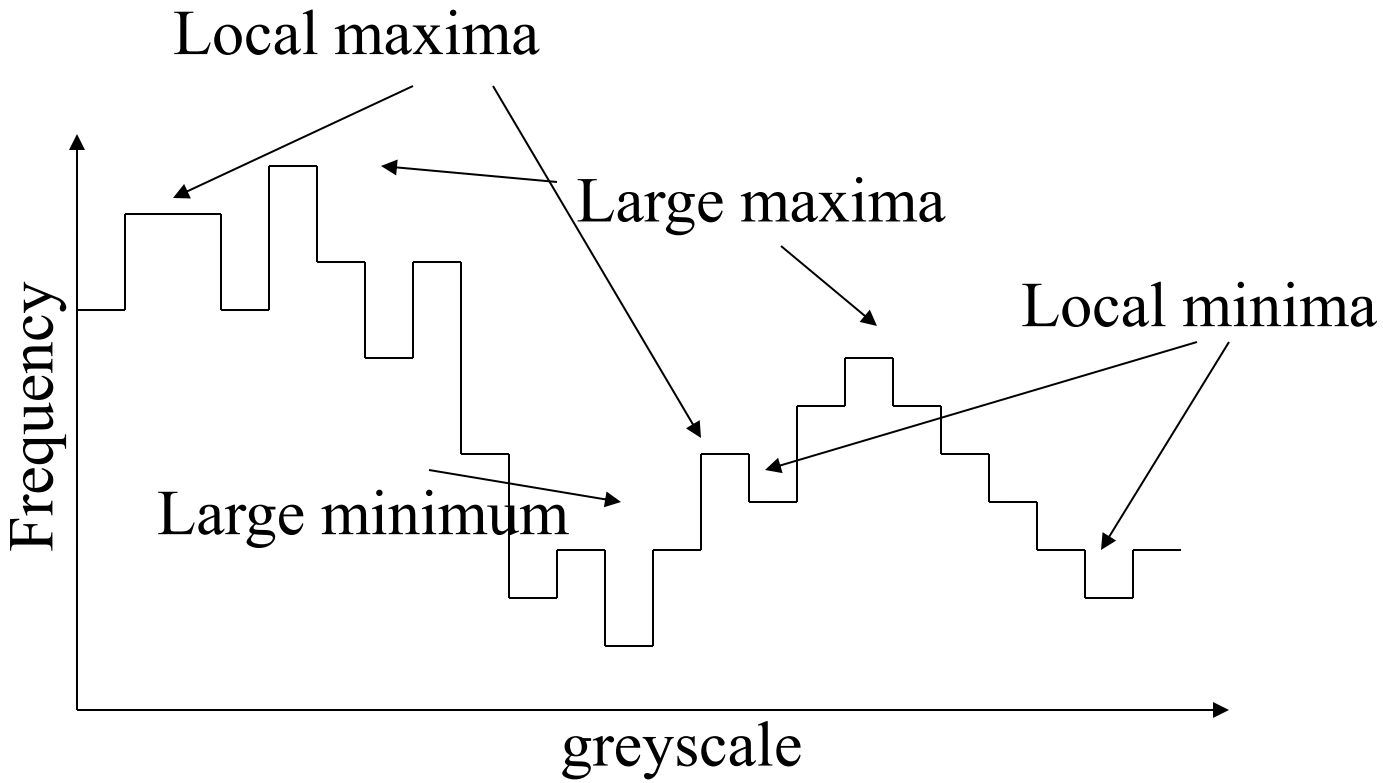
Cumulative histogram.

p = 0.3 would be used for images of this type.
For this image T = 80

Method 2: at the minimum

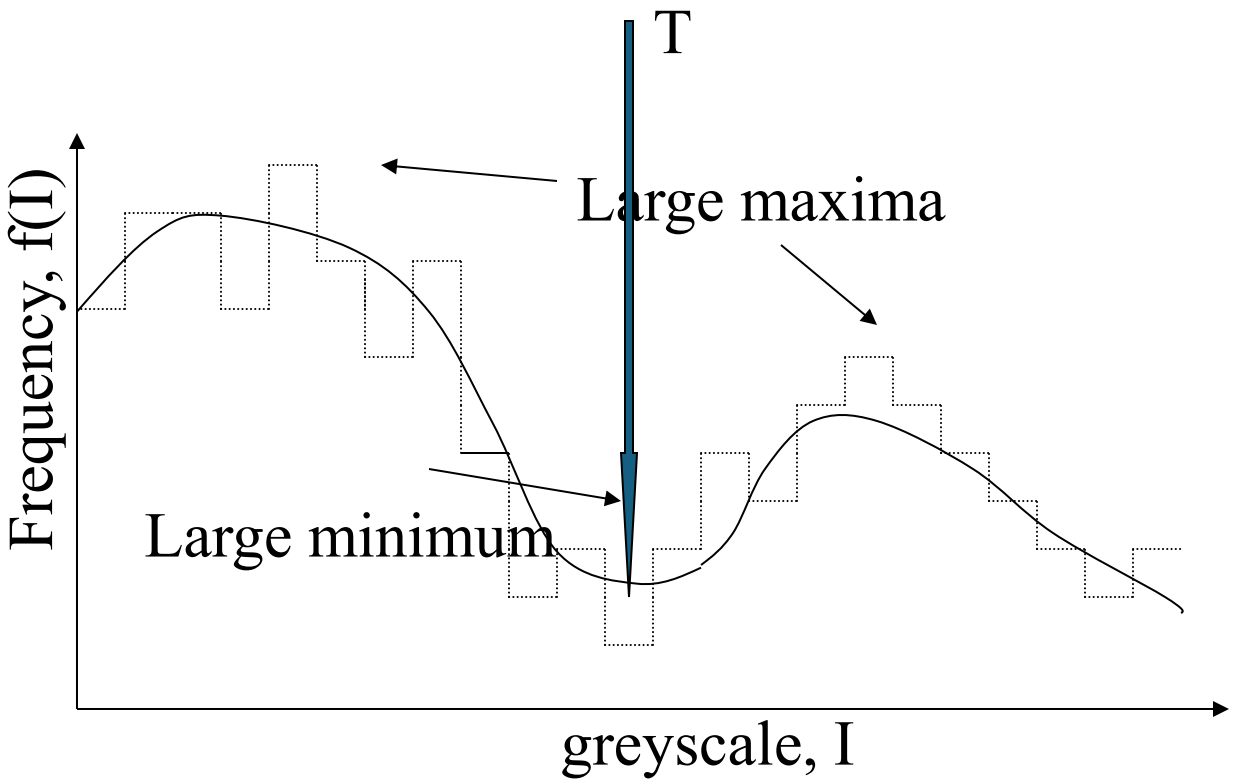
- A threshold should be located at the minimum between two maxima:
  - threshold located by identifying two maxima, and a minima lying between these
  - potential problem: local histogram variations cause local minima and maxima
  - (sometimes called “the mode method”)

Method #2: smooth minimum



Must smooth histogram to eliminate local variations using moving average filter

Smooth minimum



Threshold placed at the position of minimum between maxima

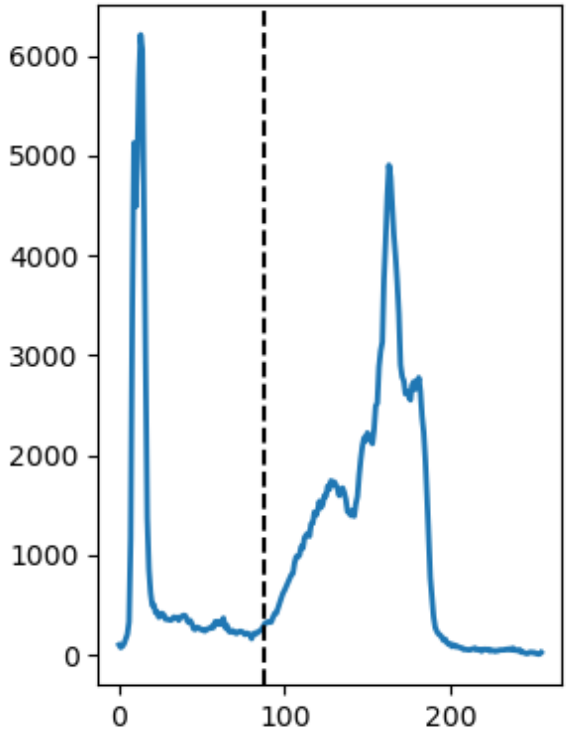
Smooth minimum

- ‘Algorithm
  - Smooth the histogram using a moving average filter
  - Starting from the left of the histogram, keep moving to the right whilst f(I) is either the same or increasing.
  - Call the point at which you stop I_1. (This will be at the left peak)
  - Starting from the right of the histogram, keep moving to the left whilst f(I) is either the same or increasing.
  - Call the point at which you stop I_2. (This will be at the right peak).
  - Find the lowest point T between I_1 and I_2. (This will be at the valley)
  - Threshold the image at T

Automatic thresholding

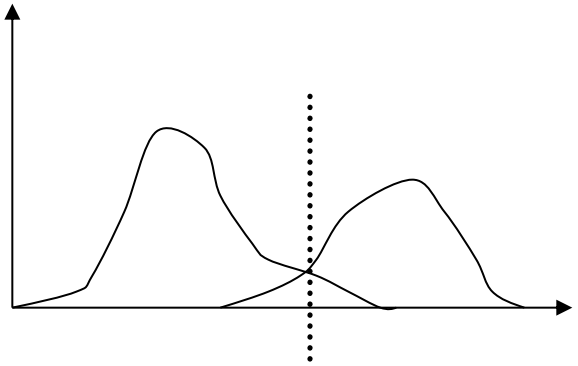






Adaptive Thresholding

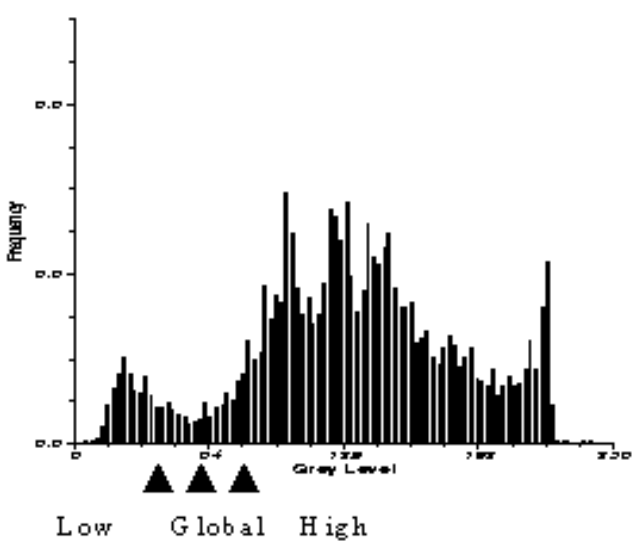
- Often background and foreground distributions are merged together.
  - distributions cannot be globally separated
  - selected a global threshold is meaningless



Background pixels wrongly labelled as foreground

Foregound pixels wrongly labelled as background

Adaptive Thresholding



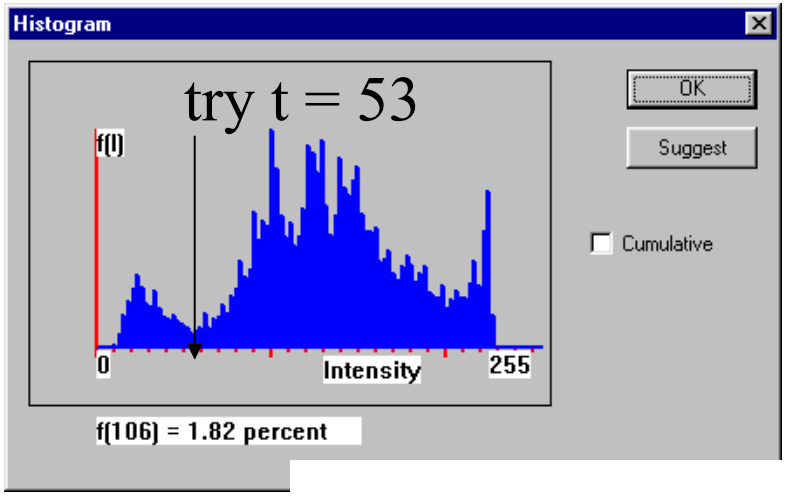


- In images where foreground objects with a wide greylevel distribution lie on a background whose intensity varies across the image, global thresholding techniques will fail.

Adaptive thresholding

- In this image the foreground and background intensities overlap.





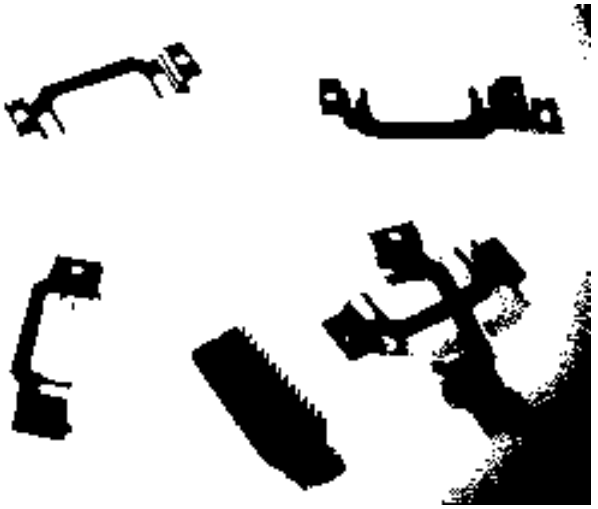
What has gone wrong in this globally thresholded image?



Adaptive thresholding

- Let's try a higher threshold

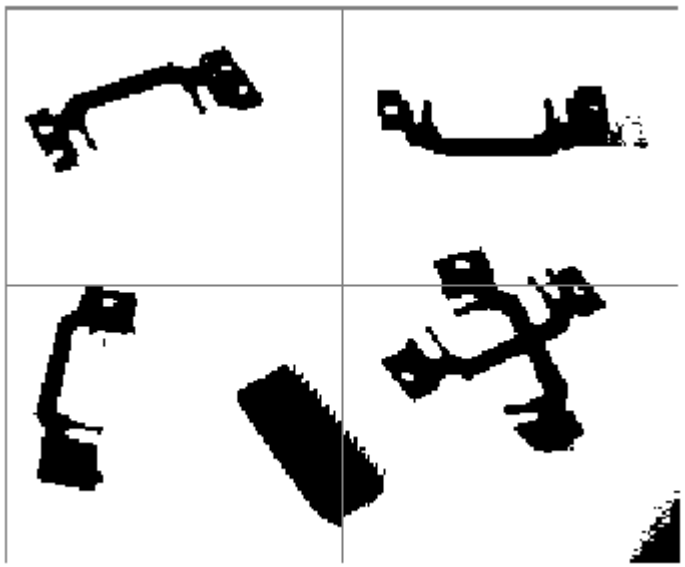


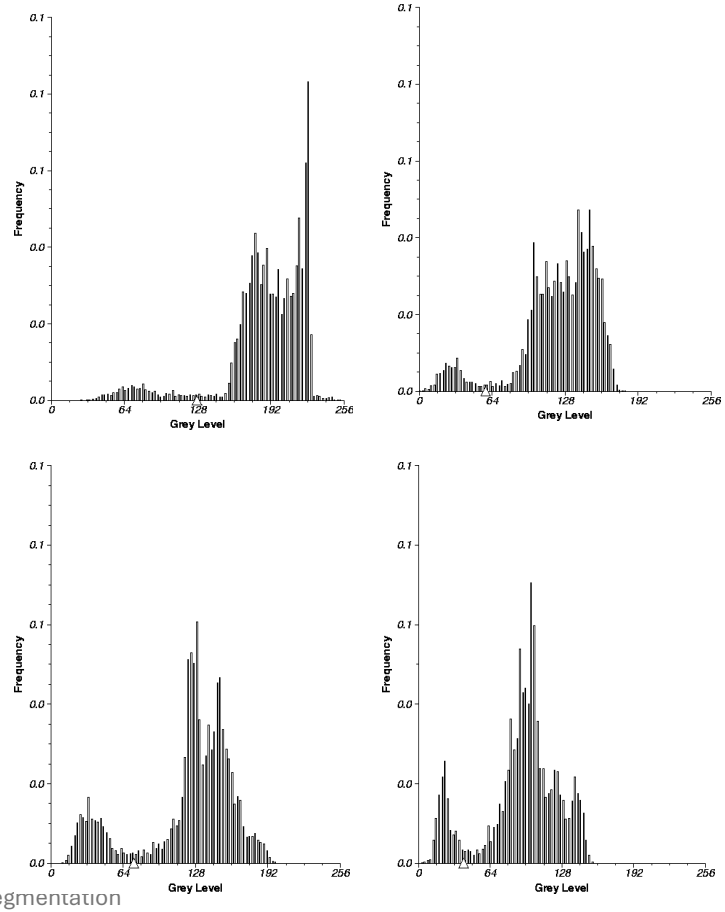


What has gone wrong now with this globally thresholded image?

Local Adaptive Thresholding

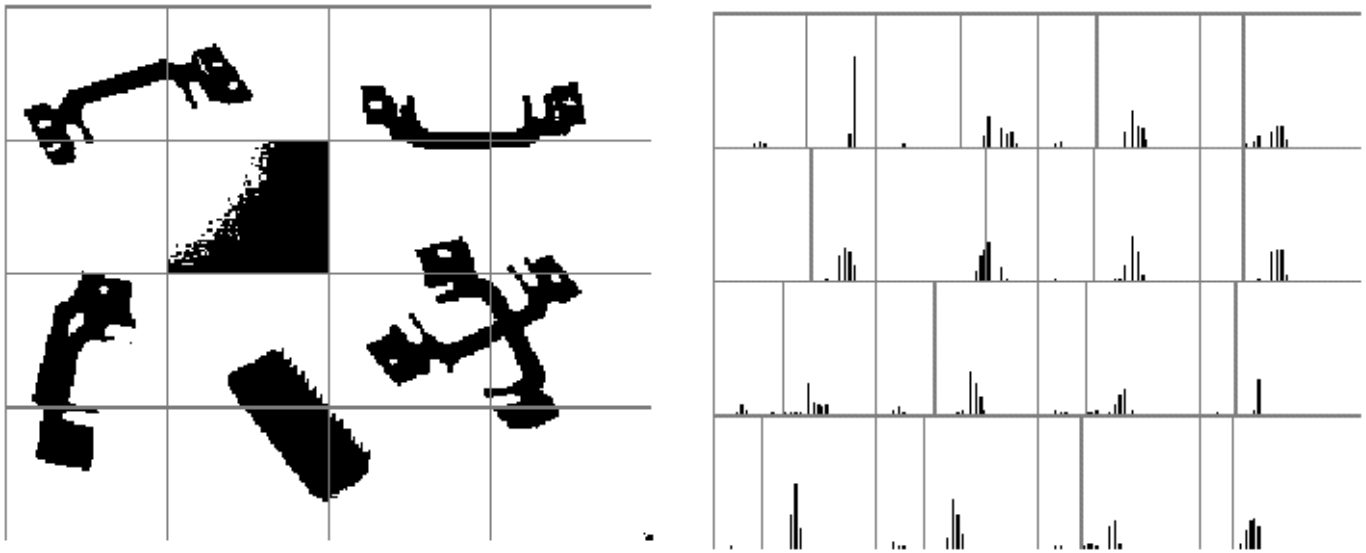
- Locally adaptive thresholding extracts a threshold for each local subimage.





In each subimage the background and foreground distributions become better separated.

Local Adaptive Thresholding



- Windows with no foreground features are thresholded meaninglessly.
- Subimage boundary effects.

Adaptive Thresholding

Instead of one global threshold, the threshold value is calculated for **each pixel** based on a **local neighborhood**.

Useful when:

- Illumination varies across the image.
- Local contrast is more important than global contrast.

Types:

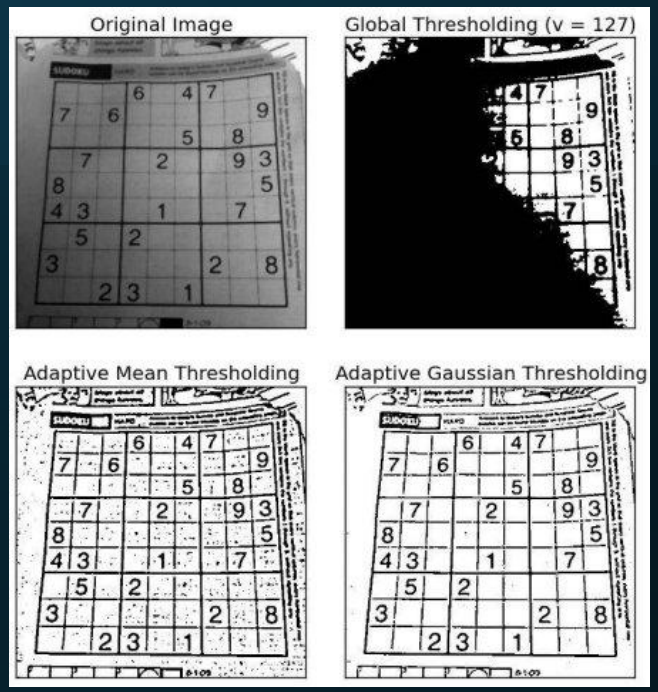
- ADAPTIVE_THRESH_MEAN_C: Mean of the neighborhood minus C.
- ADAPTIVE_THRESH_GAUSSIAN_C: Weighted sum using Gaussian kernel minus C.

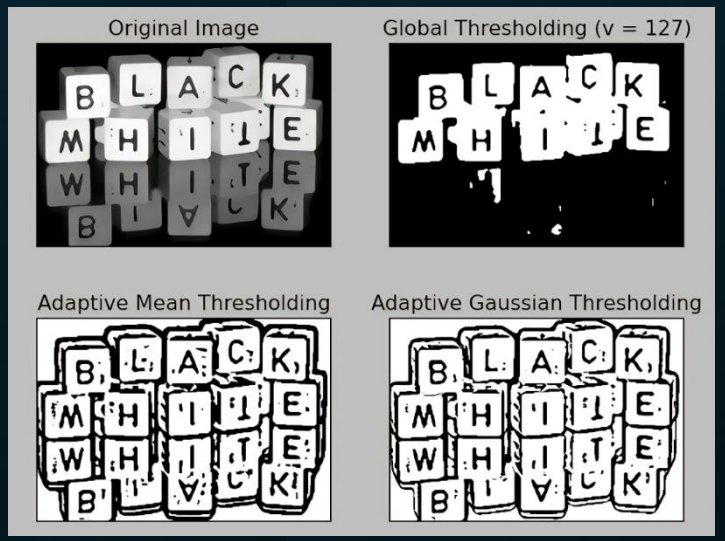
Adaptive mean and Gaussian

Adaptive Thresholding performs on an image with **uneven lighting**:

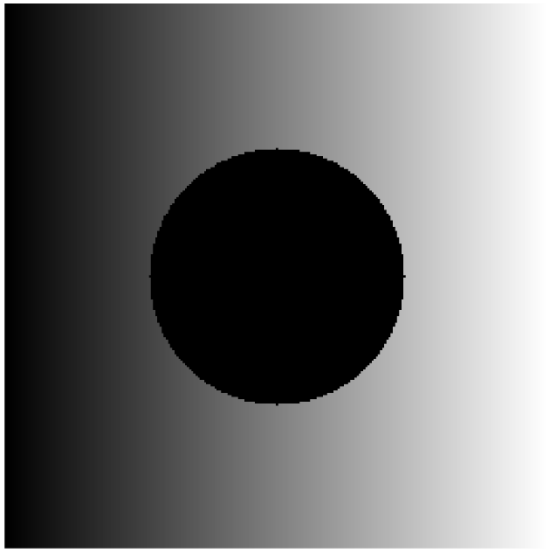
- **Adaptive Mean**: Uses local mean to detect edges and shape inside the circle.
- **Adaptive Gaussian**: Similar to mean but weighted, so the transitions are smoother.
- This method excels when lighting isn't uniform — which often breaks global thresholding.

Adaptive thresholding

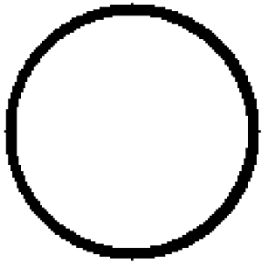




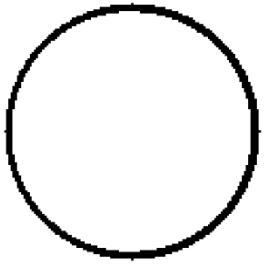
Original (Uneven Lighting)



Adaptive Mean



Adaptive Gaussian



adaptive thresholding

Image Thresholding Examples

Source picture



Fixed threshold



Adaptive threshold

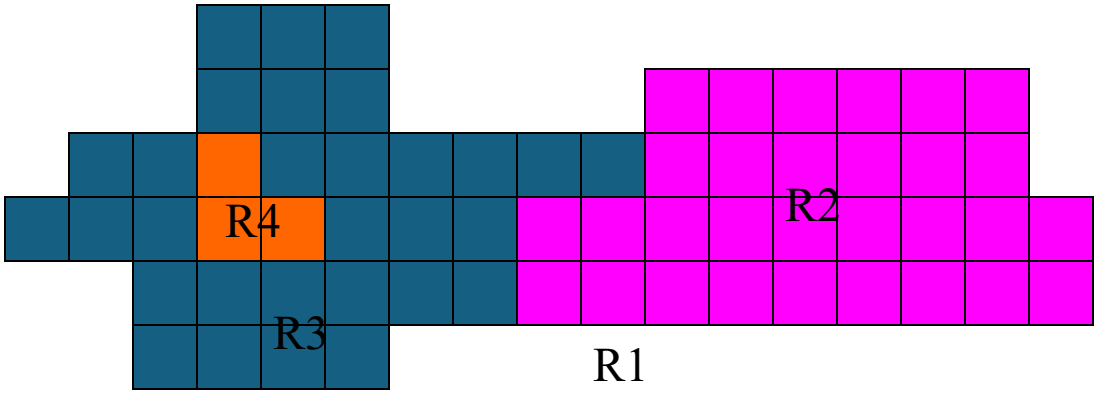


Region structure

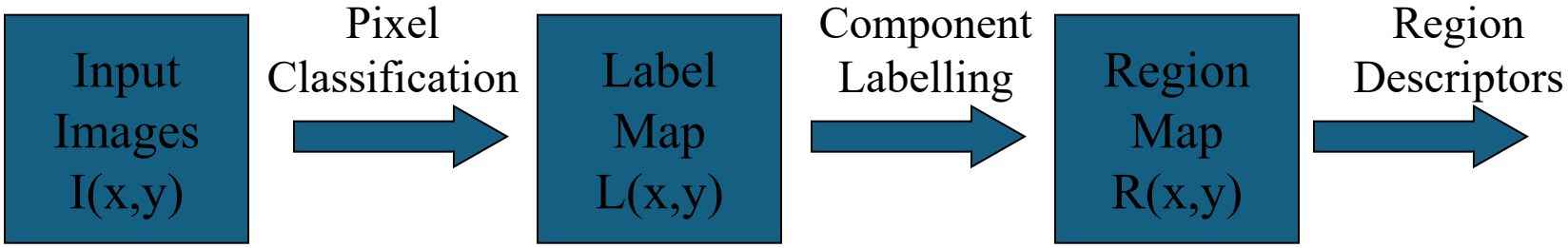
Region based segmentation

Regions are sets of immediately adjacent pixels which share some common characteristics.

For example, pixels may simply have the same greylevel, or perhaps the same local properties such as texture, or belong to same high level structure, e.g. a person.

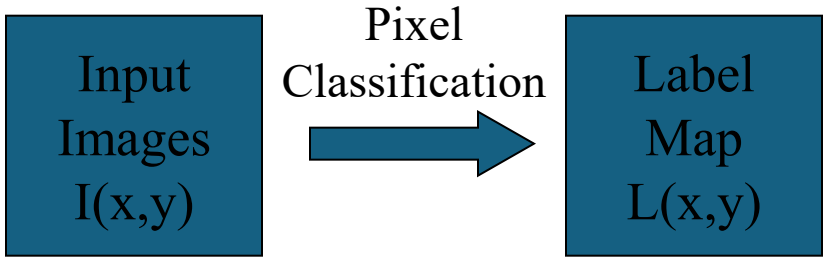


Region segmentation procedure



- LABEL MAP:
  - The pixel classification stage may be a simple stage such as thresholding (as illustrated over the page) or a more complex series of operations such as texture classification.
  - The classification may have more than one image as an input, e.g. colour classification on RGB images, or land usage based on multichannel satellite imagery.
- REGION MAP: The greylevels are used to record the region number to which a pixel belongs.
  - The component labelling stage (e.g. connected components) is used to group adjacent pixels with the same classification greylevel into unique regions.

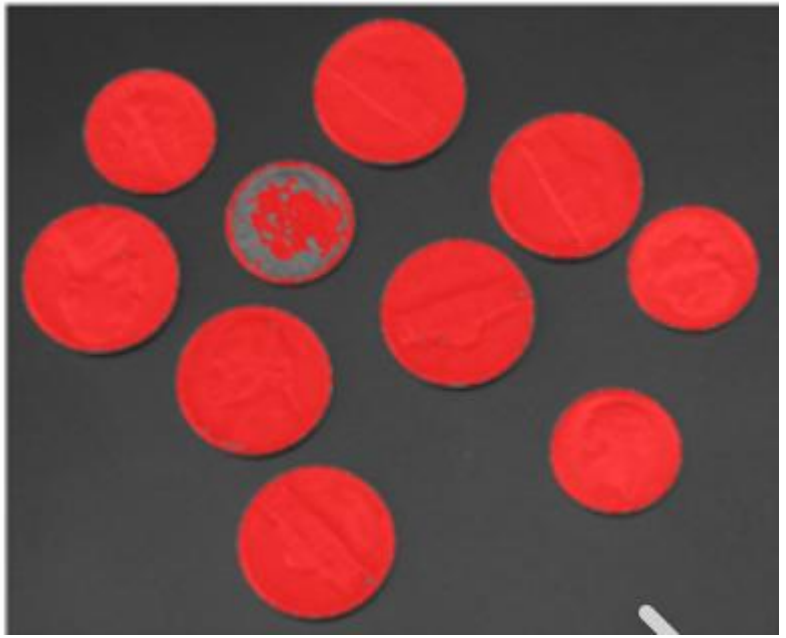
Threshold classification



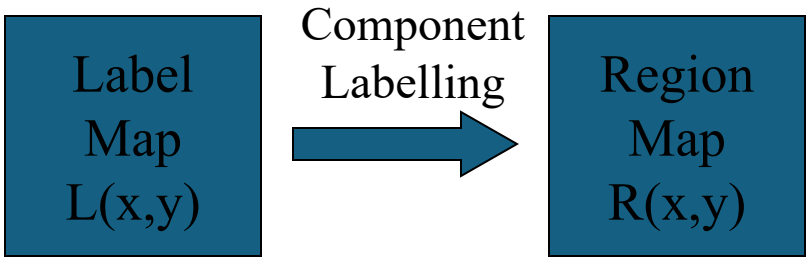
A classification process of some form is used to convert from the original image(s) to a label map.
This could be as simple as thresholding or complex segmentation.

Image segmentation (label map)





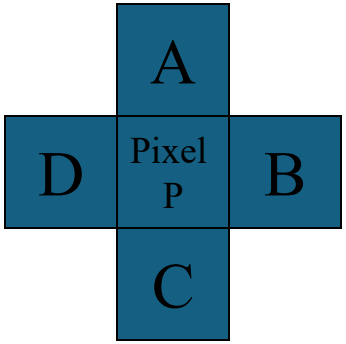
Obtaining a region map from a label map

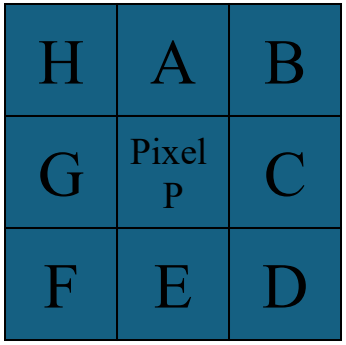


- The connected components (CC) algorithm is used to convert from a label map to a region map.

4-point / 8-point Connectivity

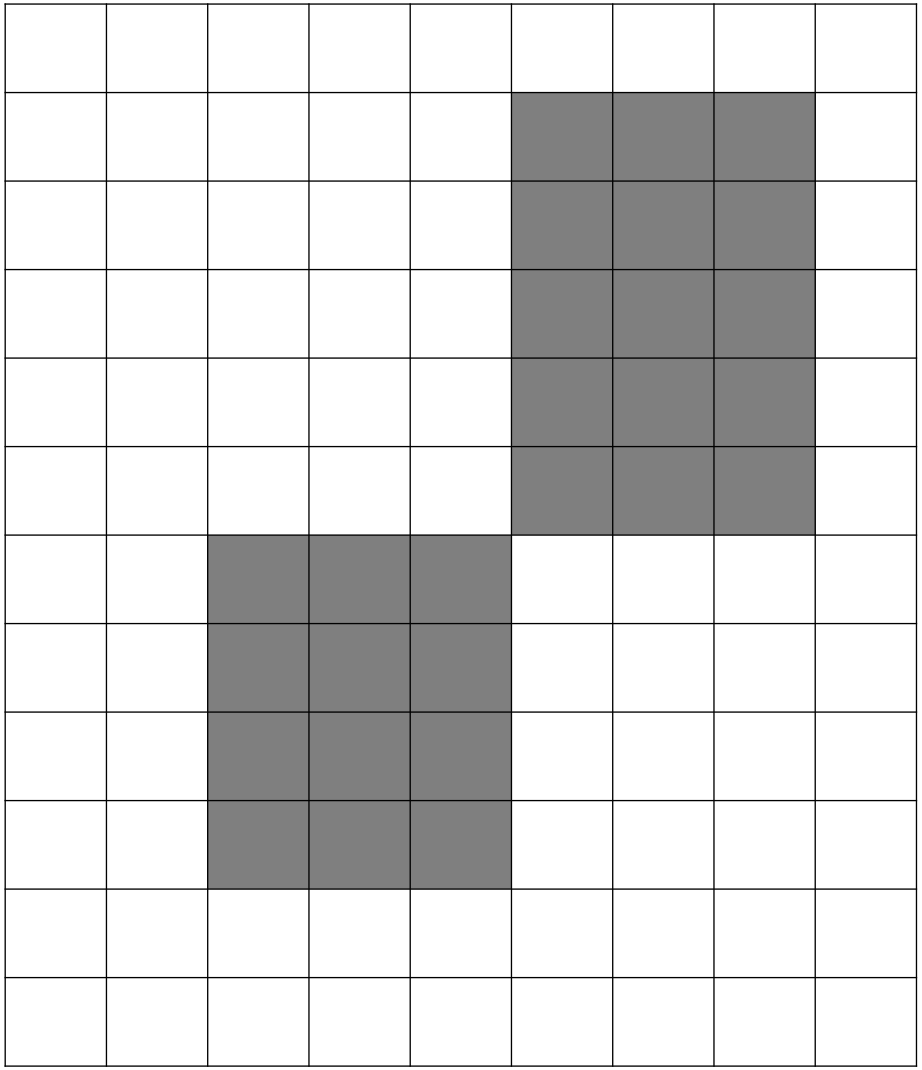
- Two different types of connectivity are defined, depending on the definition of neighbourhood (4-point/8-point)
- **Connectivity**
  - **4-connectivity:** Each pixel has up to 4 neighbors (up, down, left, right).
  - **8-connectivity:** Each pixel has up to 8 neighbors (the four above, plus the diagonals).





4-point / 8-point connectivity

- How many connect-component regions exist in the image?
- Depending on your problem, you choose the appropriate connectivity. Often 8-connectivity is used if diagonal connections are meaningful; otherwise, 4-connectivity is used.



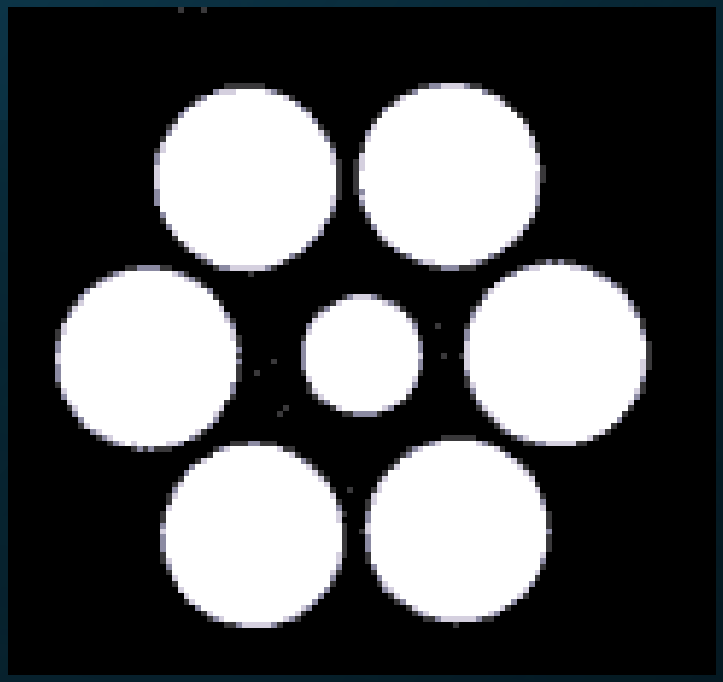
Connected Component Labelling (CCL)

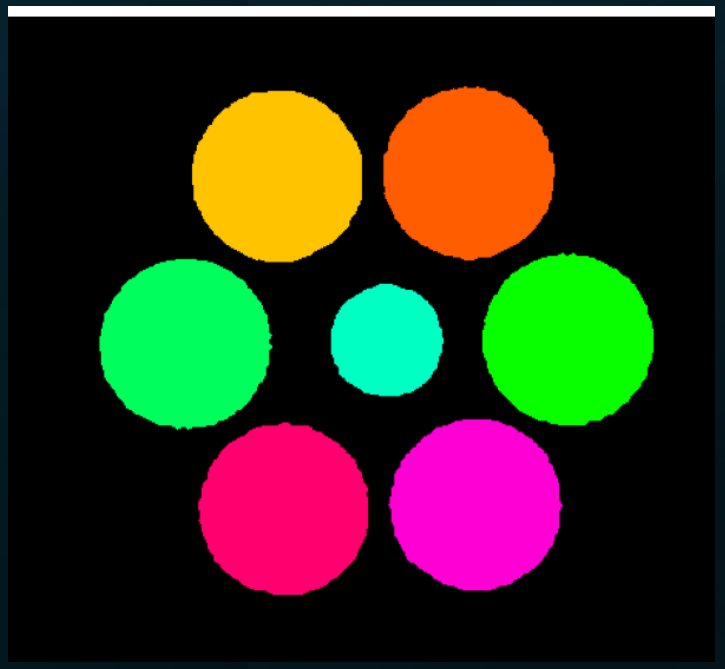
- Once you have a *segmented* (often binary) image, you typically want to label each distinct connected region.
- This procedure is called **Connected Component Labeling** (CCL) or connected component analysis.
- Each set of connected “1” pixels (foreground) forms a *component*, and we typically assign a unique label (e.g., 1, 2, 3, …) to each distinct connected region.

High-Level Steps

1. **Initialize a label counter** to 0.
2. **Initialize a label matrix** (or array) of the same size as the input image, filled initially with 0 (meaning "no label assigned yet").
3. **Iterate** over each pixel (x, y)
    1. If the pixel is foreground (e.g., 1) and its label in the label matrix is 0 (meaning unvisited):
        1. Increment your label counter.
        2. Perform a flood-fill (DFS or BFS) starting from (x, y), labeling all connected foreground pixels with the current label counter.
4. **End** when all pixels have been visited or labeled.

Component connectivity: or “Connected components” algorithm





Component labelling or “Connected components” algorithm

Note: this algorithm deletes the original label map to work out the region map so you may need to make a copy

```
int connComp(LabelMap,RegionMap,Background)
begin
        regionNumber = 0
        for i = 0 to Width-1
        for j = 0 to Height-1
        if LabelMap[i,j] != Background
                begin
                        label = LabelMap[i,j]
                        regionNumber = regionNumber + 1
                        grass(i, j,label,regionNumber,LabelMap,RegionMap)
                end
        return(regionNumber)                    (number of regions)
end
```

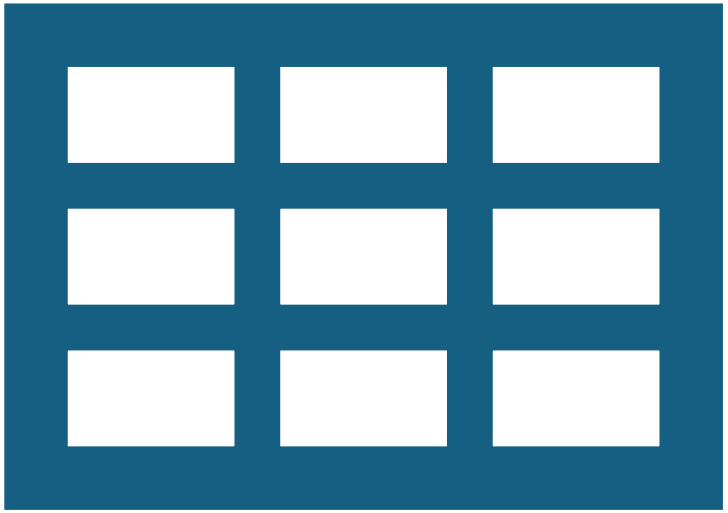
Component labelling or “Connected components” algorithm

```
grass( i, j, label, regionNumber, LabelMap, RegionMap )
begin
        LabelMap[i,j] = Background
        RegionMap[i,j] = regionNumber
        for x = i-1 to i+1
        for y = j-1 to j+1
                if (x>=0) and (y>=0) and (x<Width) and (y<Height)
                And !((x==i)&&(y==j))
                        if LabelMap[x,y] == label
                                grass(x, y, label, regionNumber,
                                        LabelMap, RegionMap)
end
```

Region data structures

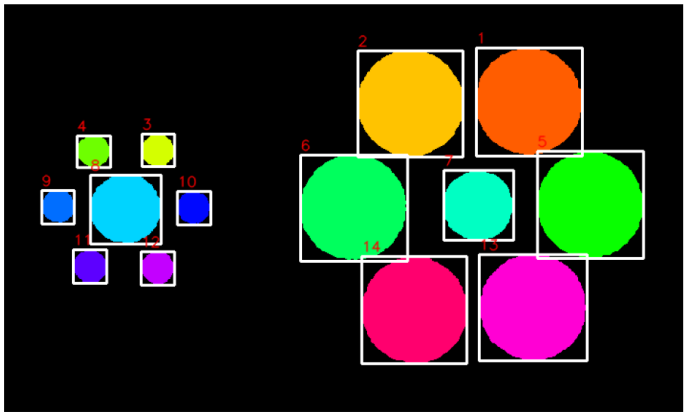
- Once identified regions can be represented in a number of different ways depending on how this data is to be manipulated.
- IMAGE DATA STRUCTURES
  - *Pseudo-coloured:* if the explicit regional relationships of pixels is once again required for human inspection.
  - *Region map:* an image is created in which each greylevel stores a unique region number.

Region data structures

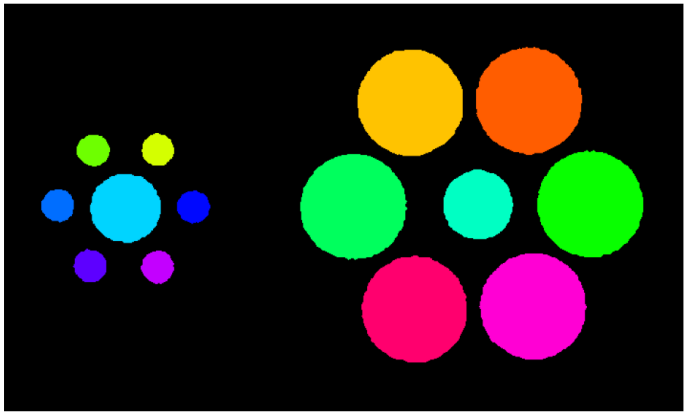


- LIST STRUCTURES
  - *Image bounds:* a list of the rectangular bounds of a region { i_{min}, i_{max}, j_{min}, j_{max} }.
    - Usually used in conjunction with the region image to efficiently process the pixels in each region.
  - *Contour lists:* a list of contour lists of region contour points. Contours may be 4-point connected or 8-point connected

4) Annotated Regions (Label Numbers)



3) Color-Labeled Regions



Region data structures

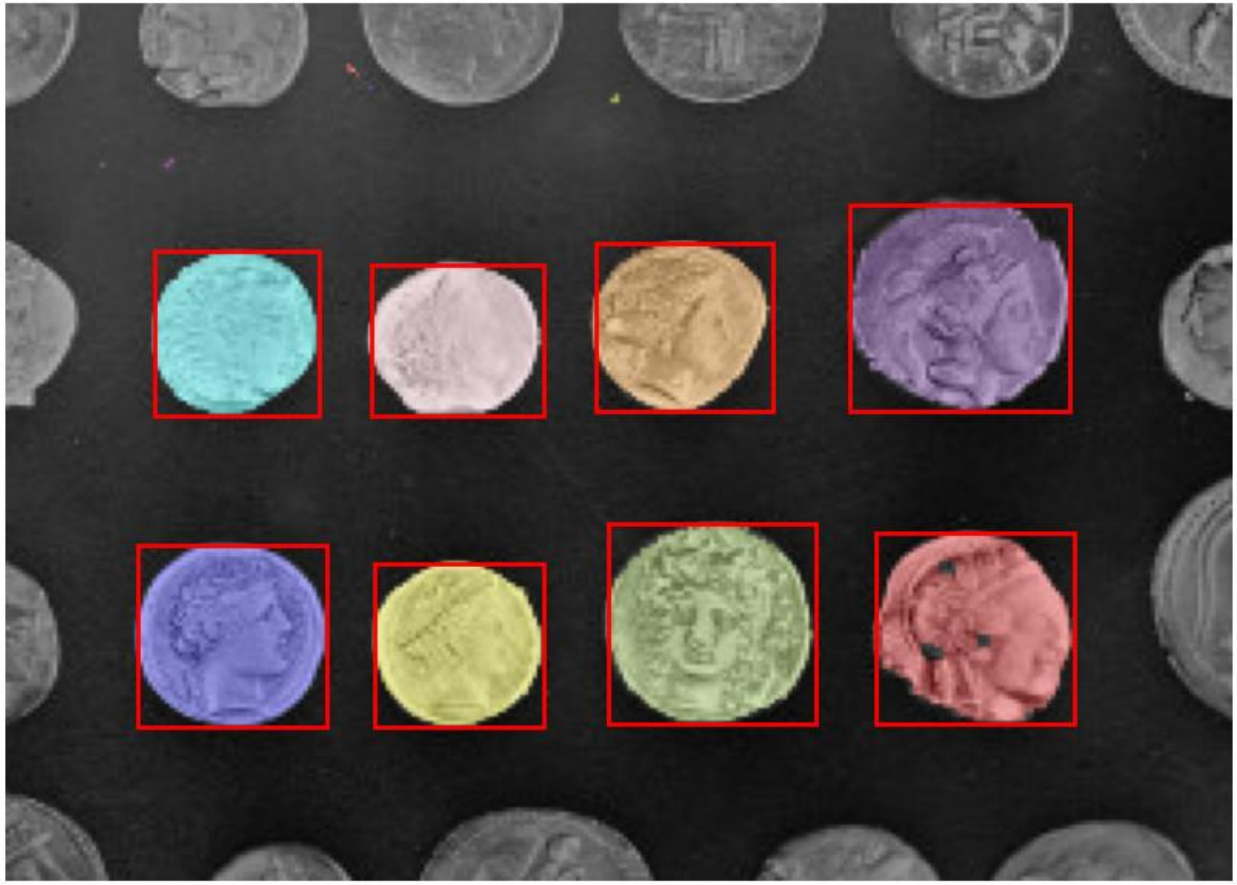
- LIST STRUCTURES
  - *Region descriptors:* a list L of sets P of region properties for each region R measuring area, centroids, average greylevel etc.

$$L = \{ P_1, P_2, P_3, ...\}$$

$$P_1 = \{area_{R1}, XC_{R1}, YC_{R1}, averageGL_{R1}\}$$

$$P_2 = \{area_{R2}, XC_{R2}, YC_{R2}, averageGL_{R2}\}$$

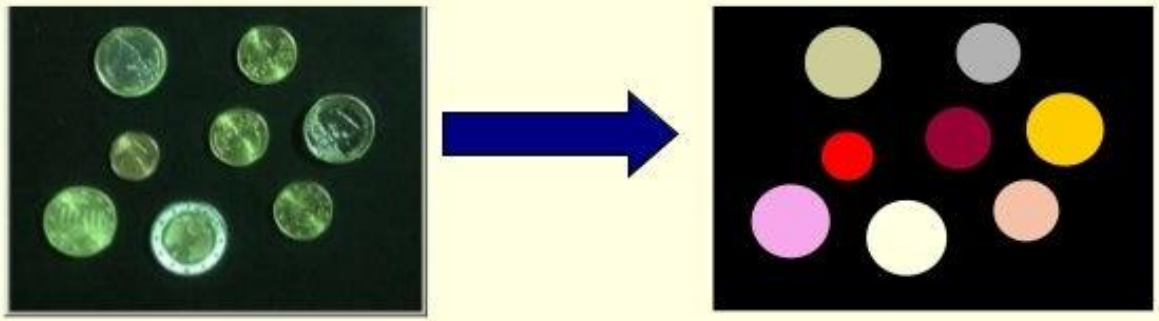
Component analysis : Region map (Image labelling)

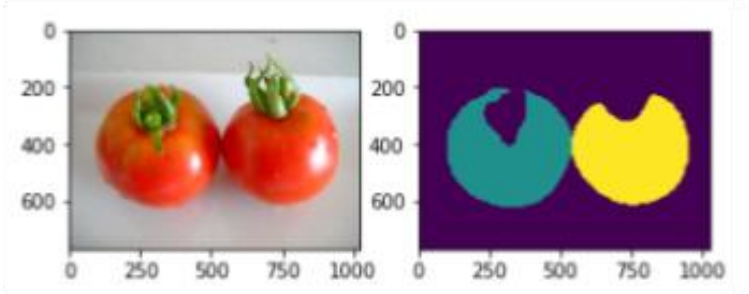


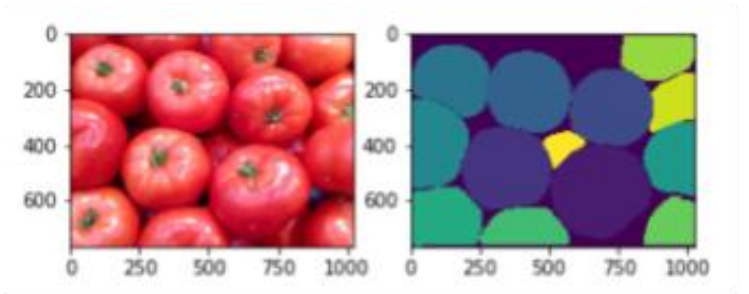
Segmentation and component analysis

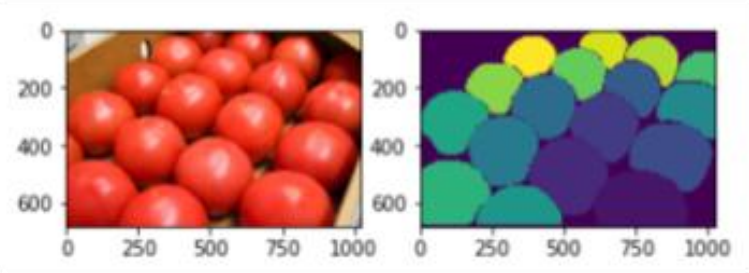
Image Segmentation

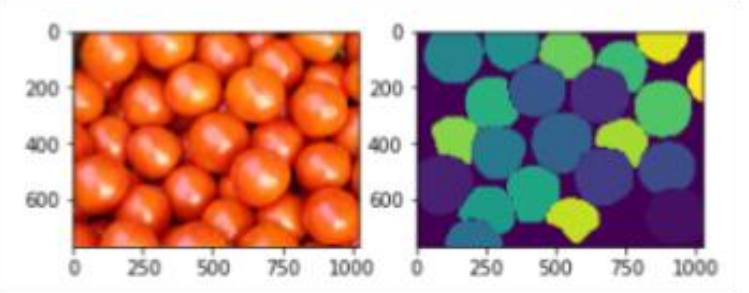
- Segmentation attempts to partition the pixels of an image into groups that strongly correlate with the objects in an image
- Typically the first step in any automated computer vision application



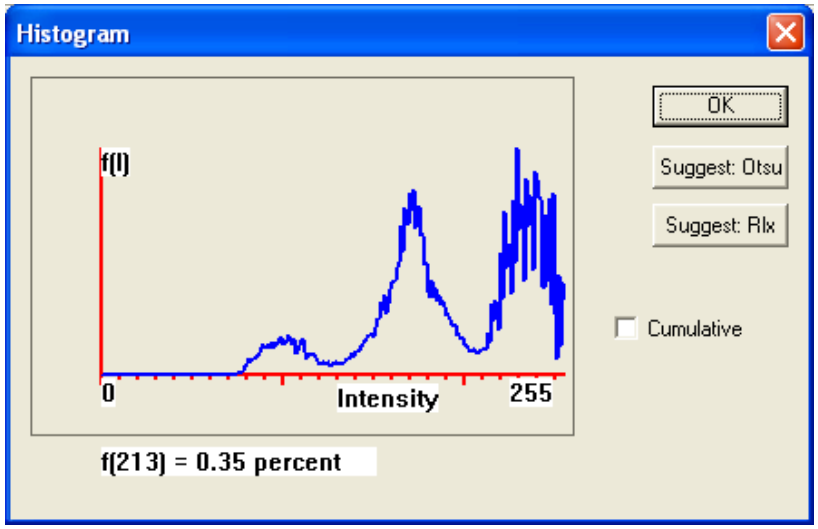

0
200
400
600
0 250 500 750 1000


0
200
400
600
0 250 500 750 1000


0
200
400
600
0 250 500 750 1000

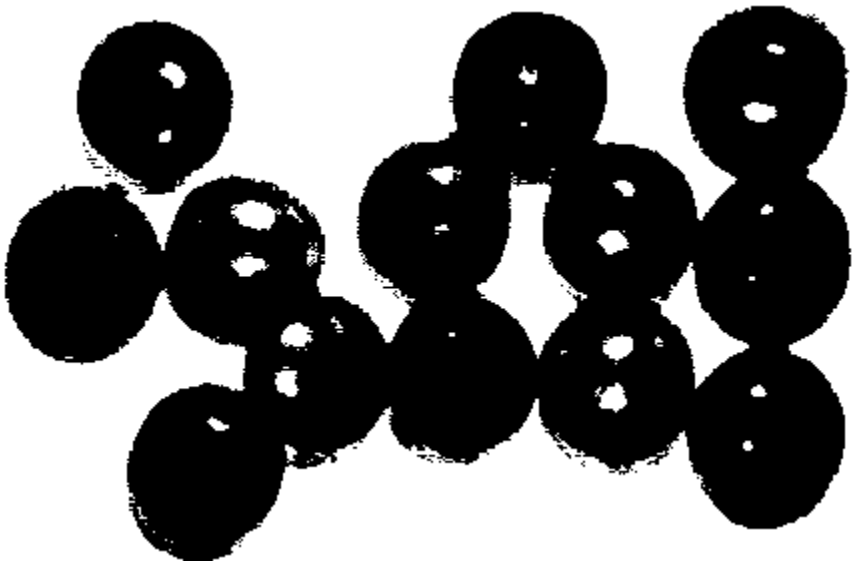

0
200
400
600
0 250 500 750 1000

Original image



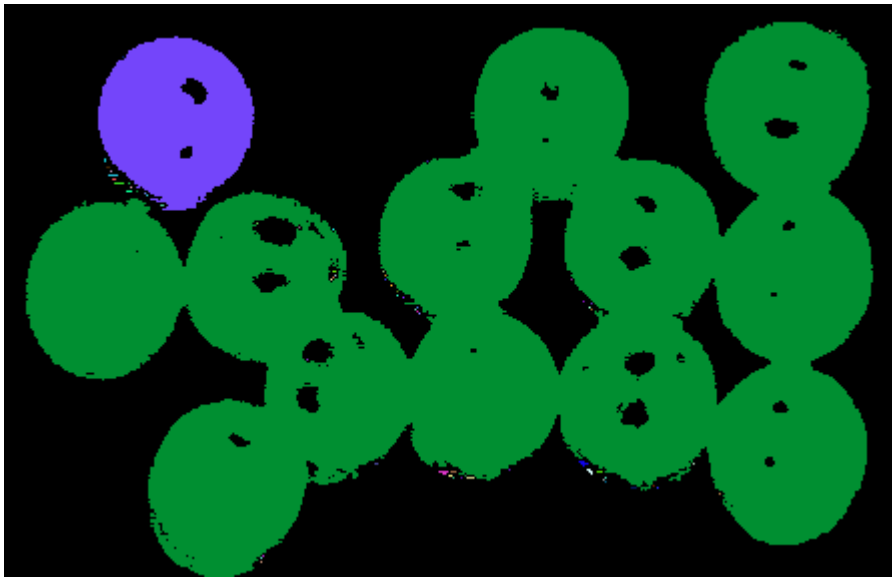


Threshold and component analysis



Label map

LABEL {Background,Feature}



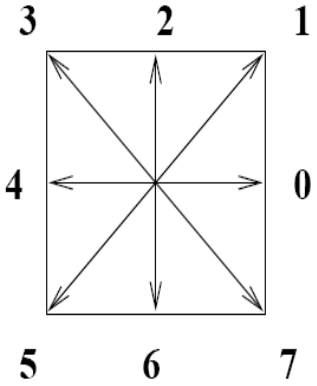
Region map

- Here the original colour image is converted to greyscale and then thresholded to obtain the label map (image segmentation. The connected components algorithm is applied to obtain the region map.

Representation of regions

- **overlay** or **mask**: binary image for each region or cluster
- **labeled image**: integer code for each region or cluster
- **boundary coding**: use perimeter set or chain code
- **quad tree**: hierarchical spatial partition with white,black,grey nodes
- **property table**: property or feature vector

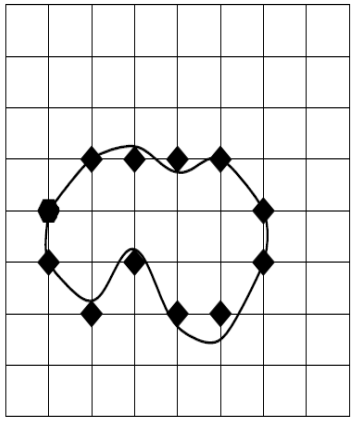
Chain codes for boundaries



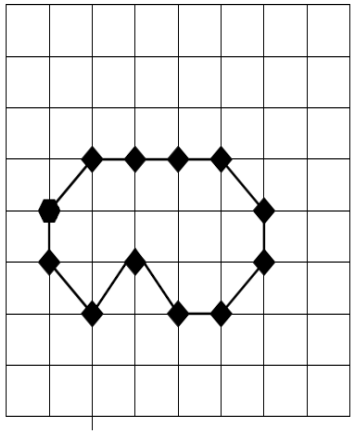
encoding scheme

100076543532

chain code representation



original curve



chain code links

Quad trees divide into quadrants

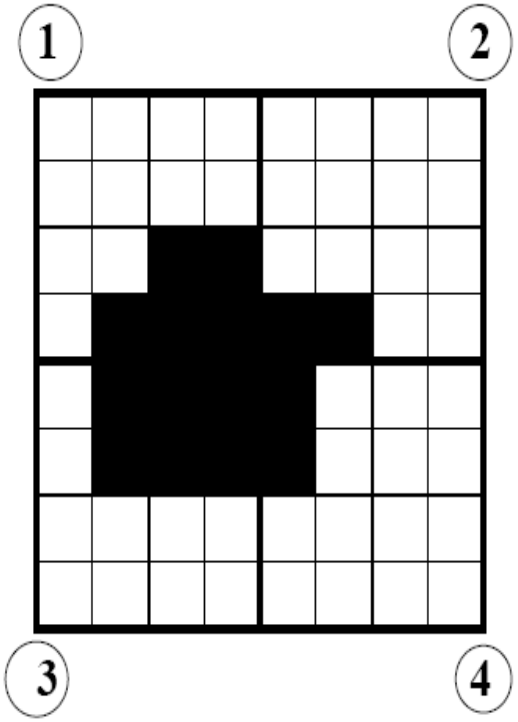
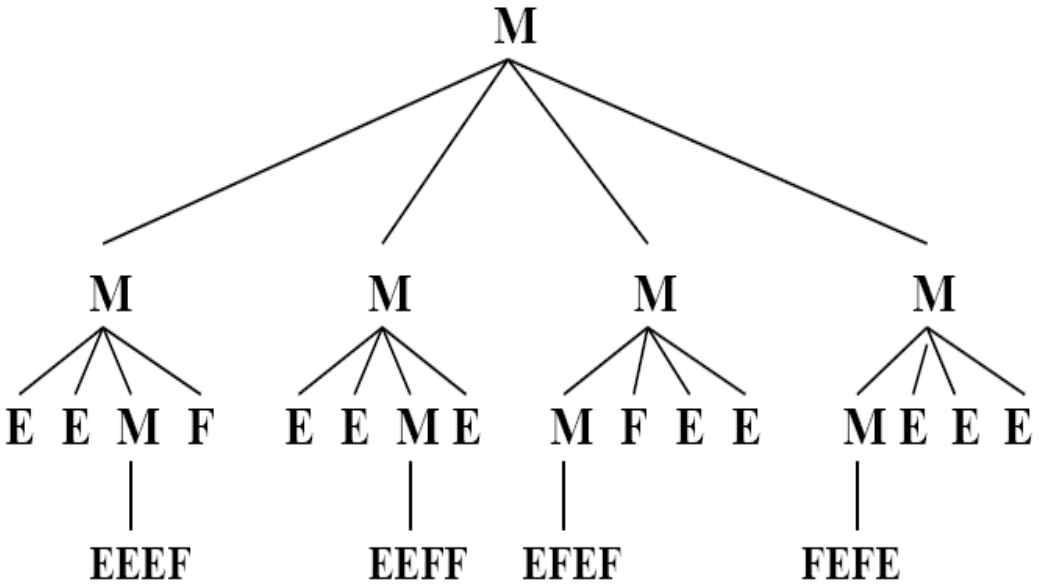


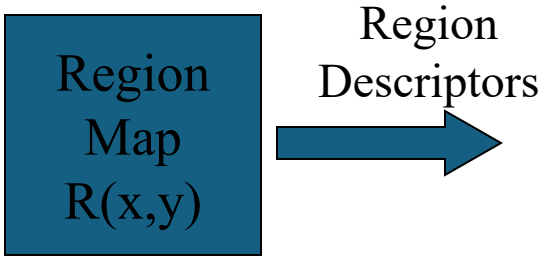
image region



M=mixed; E=empty; F=full

quad tree representation

Region Descriptors/ Feature Extraction



- From the region map, higher level descriptors such as region lengths and areas are obtained.

Region area

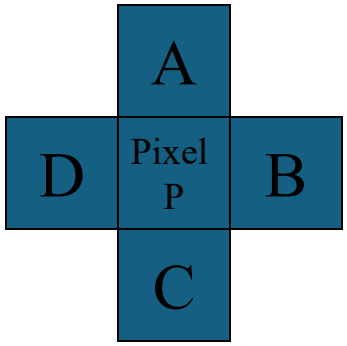
- Recovering region area is fairly straightforward.
  - Set up a 1D array the size of the number of regions in the image, and initialise to zero.
  - Go through the image in a raster scan fashion and increment the array element which has the same index as the pixel value.
  - The array elements now show region number against area.

Region Number	Area
0	100
1	255
2	322

Region Length

- Measuring the **“length”** of a region in a digital image can refer to several related concepts.
- Often, the most common interpretation is to measure the **perimeter** (the boundary length) of that region.

Recovering region boundaries

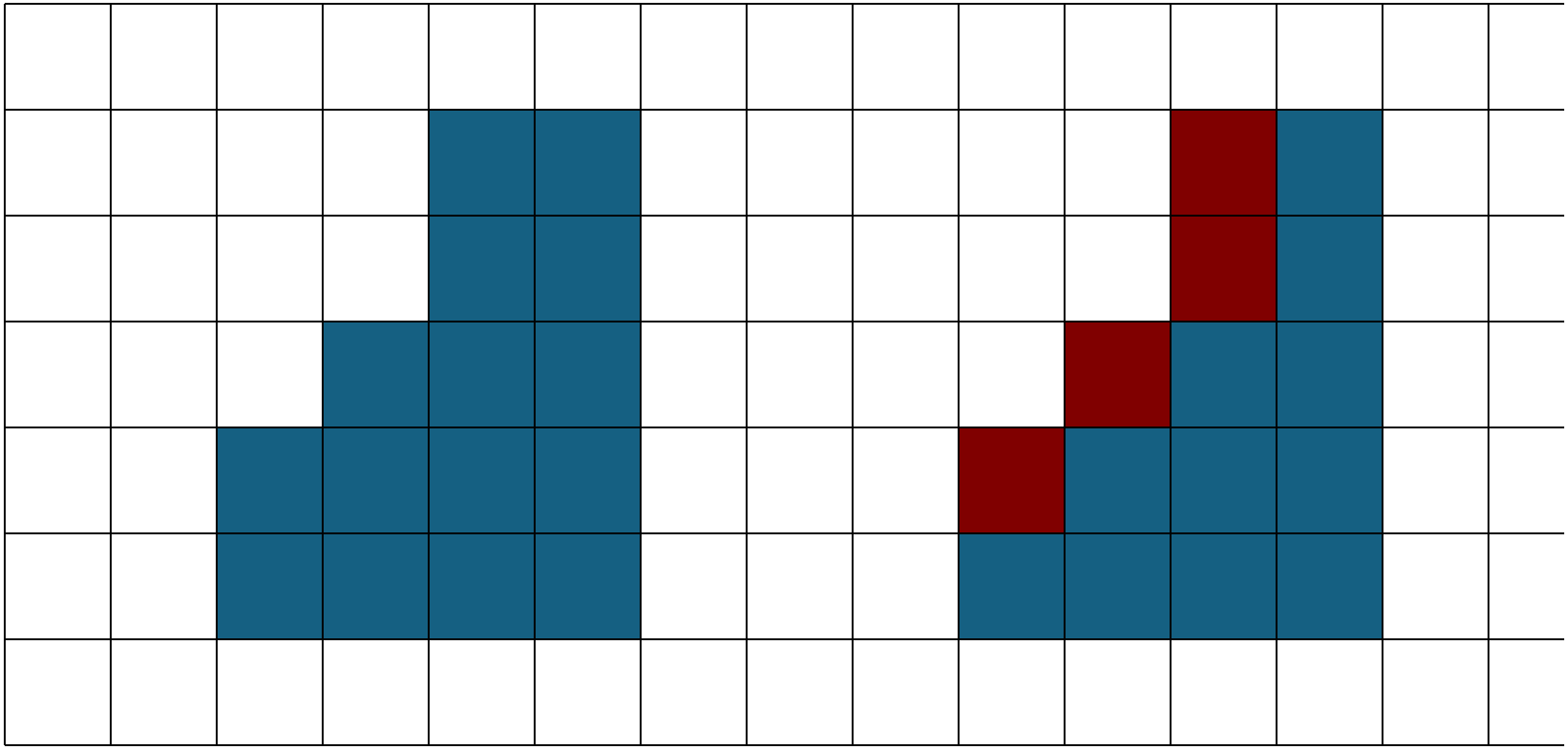


"star operator" applied for 4-point boundary connectivity

if any pixel A,B,C,D has a different greylevel than pixel P, then mark it a region boundary.

```
for every pixel i,j
if L[i,j] not background then
        if ((L[i,j]!=L[i+1,j]) or (L[i,j]!=L[i,j+1]) or
                (L[i,j]!=L[i-1,j]) or (L[i,j]!=L[i,j-1)) then
                        Edge[i,j] = 1
                else
                        Edge[i,j] = 0
                end if
end if
```

Recovering region boundaries



Region length

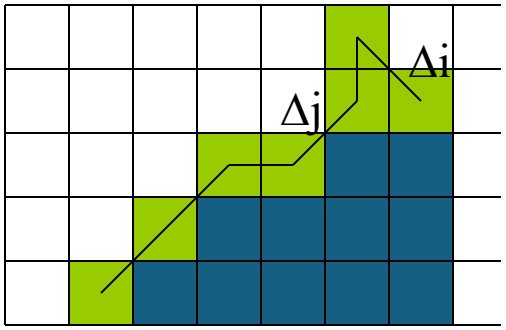
- Recovering region length is also fairly straightforward.
  - Set up a 1D array the size of the number of regions in the image, and initialise to zero.
  - Go through the image in a raster scan fashion and increment the array element which has the same index as the pixel value **AND** which is an edge. This will give you an **APPROXIMATE** estimate of the length.
  - The array elements now show region number against length.

Region Number	Length
0	10
1	244
2	23

Describing Region Shape

- **Area:** The number of pixels belonging to a region. Assumption: given a specific camera at a predefined distance from the work bench, all objects of the same type will project onto regions of roughly the same area in pixels.
- **Perimeter:** An approximation of the perimeter length can be computed from the number of boundary pixels located by the “star” operator.

$\text{sqrt}((\Delta i)^{2+}(\Delta j)^2)$

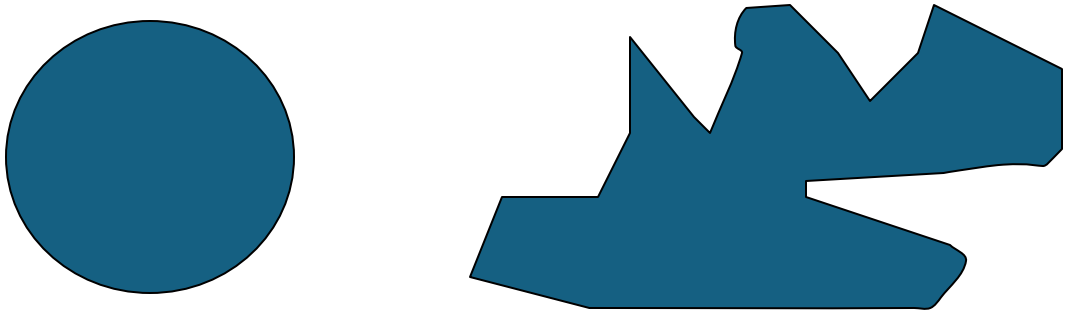


Problem: the real perimeter is the sum of the gaps between the border pixels which may be 1 or sqrt(2)

Describing region shape

- Compactness: define compactness, C, in terms of perimeter P and region area A as

$$C = P^2 / (4 \pi A)$$



- Most compact shape CIRCLE has compactness = 1
- An uncompact shape has compactness >> 1
- (some books define this differently!)

Shape matching

- **Example:** after region analysis a set of detected objects have the following lengths and areas:

Region Number	Length	Area
0	31	79
1	20	25
2	33	78

- **Question:** Which is a circle? Which one is a square shape. Which two are similar?